

Universal Quantification for CUE

Semantics and certification of open constraints

Aram Hăvărneanu

28 September 2026

Working technical proposal — v9

Abstract

CUE descriptions specify possible completions of shared subjects. This proposal extends that interpretation with higher-rank impredicative universal quantification: one subject satisfies a predicate at every semantic type instance. Function types are universal predicates on successful application, with contravariant inputs and covariant results. Certification establishes these predicates for open implementations using proved bounds on live references. Evaluation and certification jointly propagate information through a scoped constraint graph; exact residuals preserve their dependencies under subsequent unification. A relevant source discipline checks the observations explicitly claimed by interfaces and distinguishes persistent refutation from pending obligations. The development establishes substitution, conservativity, semantic soundness, progress and preservation for partial computations, and conditional failure-free composition. A finite mandatory checking kernel, explicit proof support, and accounted work limits specify how these guarantees are obtained.

1 Introduction

CUE treats data and schemas as descriptions ordered by refinement. A declaration constrains the possible completions of a subject; another declaration about the same subject contributes another constraint. Unification is their conjunction. This interpretation gives configuration composition its central invariant: additional information restricts a result rather than replacing it.

Polymorphic interfaces express an obligation of a different logical shape. An ordinary field may be completed by a suitable value. A universally quantified field must admit one value that satisfies every instance of its description. Applying this distinction to operational closures yields function contracts: one implementation must respect the stated input–output relation at every admissible input and every type instance. Universal quantification can therefore supply higher-order interfaces while conjunction keeps its meaning as the intersection of predicates on a shared subject.

The surrounding configuration remains open. A type expression can refer to a field whose description is refined later, and a function body can capture a value that is only partially known. A useful certificate must consequently state its assumptions about those references. An established upper bound can justify an operation before its operands are concrete, while the original references continue to determine the operation’s constraints. Further unification strengthens the assumptions, supplies new proof premises, and creates any additional obligations imposed by the new declarations.

This leads to a single organization of checking and execution. The source graph retains exact constraints and their scopes. Finite certificates justify facts about that graph and the behavior of its implementations. Evaluation can compute premises needed by certification; certification can enable further evaluation. Each step preserves the source relation, and unfinished work remains an explicit residual. The same organization supports separately checked components, live function interfaces, and later unification of evaluated source.

The semantic guarantee is partial correctness: successful returns satisfy their contracts. The source discipline additionally checks declared capabilities and explicitly stated observation regions. Operational safety excludes unsupported eliminations, and composition preserves success when its components and call constraints satisfy the corresponding success premises. These distinct judgments give precise content to a certified configuration without identifying an unresolved obligation with a failed computation.

2 Semantic foundations

2.1 Descriptions and joint completions

Let D be the set of complete data of the base language. A description denotes a subset of D ; a literal denotes a singleton. Semantic refinement is inclusion: $A \leq B$ means $\llbracket A \rrbracket \subseteq \llbracket B \rrbracket$. The interpretations of bottom, top, unification, and disjunction are

$$\llbracket \perp \rrbracket = \emptyset, \quad \llbracket \top \rrbracket = D, \quad \llbracket A \wedge B \rrbracket = \llbracket A \rrbracket \cap \llbracket B \rrbracket, \quad \llbracket A \vee B \rrbracket = \llbracket A \rrbracket \cup \llbracket B \rrbracket.$$

These correspond to $_|_$, $_$, $_$, $_$, and $_$, with the orientation of CUE's subsumption order [1]. Semantic types are *extensional*: equality depends on their inhabitants.

An open description has a relational interpretation. With exposed coordinates E and local coordinates L , write $P \subseteq \text{Env}(E) \times \text{Env}(L)$ for its joint completions. Local hiding is existential projection:

$$\text{hide}_L(P) = \{w \mid \exists \xi. P(w, \xi)\}.$$

Thus ordinary values are existential in the following precise sense: completing a declaration chooses witnesses satisfying its constraints. A reference can share a witness, whereas copying a schema creates fresh cells with rebinding of its internal references [2]. The distinction is expressed by

$$\exists x. (P(x) \wedge Q(x)) \quad \text{and} \quad (\exists x. P(x)) \wedge (\exists y. Q(y)).$$

The former uses one completion; the latter permits two. Consequently, fields and their dependencies are combined before local witnesses are hidden.

A record predicate specifies field contents together with a presence policy. An open record permits additional fields; an exact record fixes its allowed labels. An optional field permits absence. Empty optional contents therefore force absence, whereas empty required contents refute the record. Field provenance, lexical scope, and the preference information carried by defaults belong to the elaborated description and its observation policy. The predicate of completions is its refinement component; preference selection follows the base rules [3].

2.2 Operational subjects and live references

Fix a set V extending D with complete finite records, lists, argument packets, and represented closures. A packet records ordered arguments, labeled arguments, and omission information. A complete closure is represented by (o, γ, β) , where o is a stable code origin, γ its free captured environment, and β its normalized bindings of partially supplied slots. The code and captures determine its calling protocol and executable defaults. Let $F \subseteq V$ be the closure inhabitants, and set $\mathcal{U} = \mathcal{P}(V)$. The sort `Type` classifies semantic predicates in $\mathcal{U}$. Write `number`, `string`, and `bool` for the numeric, string, and Boolean predicates.

Closure identity is equality of these operational descriptions. Copying, embedding, and unrelated record refinement preserve the code origin and free captures. Distinct captures distinguish closures produced by the same code. The representations are finite; equality of complete descriptions is structural. Symbolic capture equality is retained as a constraint on their completions. This gives extensional predicates on inhabitants with intensional identity.

An exposed interface E includes both data coordinates and live description coordinates. Its assignments form a set $\mathcal{W}_E$ of *worlds*, subject to the graph's lexical scopes and reference interpretation. A data coordinate is a completion witness. A description coordinate is a predicate constraining such witnesses. Source fields may name, compute, and refine these predicates. A scoped reference r retains its source node and its copy/rebinding map; write τ_r for the type expression denoting that reference.

At a world w , and a valuation ρ of type variables, the reference denotes $\llbracket \tau_r \rrbracket_{w, \rho} \subseteq V$. For example, a live field bounded by `int` imposes $\llbracket \tau_r \rrbracket_{w, \rho} \subseteq \text{int}$, where $\text{int} \subseteq V$ is the integer predicate. A lexical alias of `int` denotes the fixed predicate `int`. A successful singleton field fixes its description to that singleton. Computed descriptions retain the equations and operand demands that determine them. This interpretation keeps first-class descriptions and their future refinement in the same scoped graph.

Assumption 2.1 (Base semantic interface). *The base elaboration assigns each scoped graph its predicate of successful joint completions. Source conjunction is intersection after scope alignment. Reference interpretation commutes with capture-avoiding substitution and pullback along context extension. Fresh local witnesses are hidden after sharing is established. Legacy constructors, reference rules, and preference selection retain their base interpretations.*

Adding a declaration restricts this predicate on a common interface. The interpretation of an existing reference is fixed; its admissible worlds become more constrained. Observation may project away description coordinates, while source intended for further unification retains the coordinates that later declarations may address.

Operational application is fixed on represented closures independently of the valuation of erased type variables. Write $f p \Downarrow v$ for a successful return on a complete packet. A symbolic implementation selects a completed closure f_w in each world. Its data captures and represented assertions are interpreted in that world. All subsequent semantic equations are pointwise in w ; the subscript is suppressed where no live reference occurs.

2.3 Type expressions

Types are finite expressions generated by

$$\begin{aligned} T, U ::= & \perp \mid \top \mid \alpha \mid \tau_r \mid k \mid c \mid T \wedge U \mid T \vee U \mid T \rightarrow U \\ & \mid \{\ell_i^{m_i} : T_i\}_{i \in I} \mid [T_1, \dots, T_n] \mid \text{List}(T) \mid \forall \alpha \leq U. T. \end{aligned} \quad (1)$$

Here k is a base kind or a represented closed first-order data predicate, c is a literal, and m_i records field presence. A record also carries its open or exact label policy. Acyclic aliases expand by capture-avoiding substitution. The unbounded form $\forall \alpha. T$ abbreviates $\forall \alpha \leq \top. T$. A bound may use earlier type variables; its binder scopes over the body only.

A function interface is formed in its enclosing graph and type-variable scope. Its term parameters bind the function body. Scope checking follows references transitively, including aliases and rebinding maps. The same rule applies to a written type and to a live description used in its place.

Base predicates have their fixed interpretations, $\llbracket \alpha \rrbracket_{w, \rho} = \rho(\alpha)$, and the lattice operations denote intersection and union in $\mathcal{U}$. Records and tuples impose their component predicates. $\text{List}(A)$ contains the finite lists with every element in A , including the empty list. Bottom is the empty predicate. Universal and arrow types receive the interpretations below.

2.4 Quantifiers on a fixed subject

For a context set X , predicates form the inclusion-ordered fiber $\mathcal{P}(X)$. A context map $h : Y \rightarrow X$ acts by inverse image $h^*P = h^{-1}(P)$. For a projection $\pi : X \times I \rightarrow X$, define

$$\exists_\pi P = \{x \mid \exists a \in I. (x, a) \in P\}, \quad (2)$$

$$\forall_\pi P = \{x \mid \forall a \in I. (x, a) \in P\}. \quad (3)$$

The subject being described belongs to X . Quantification removes the bound coordinate while keeping that subject fixed.

Proposition 2.2 (Adjunction and reindexing). *For $P \subseteq X \times I$ and $Q \subseteq X$,*

$$\exists_\pi P \subseteq Q \iff P \subseteq \pi^*Q, \quad \pi^*Q \subseteq P \iff Q \subseteq \forall_\pi P.$$

*For $h : Y \rightarrow X$, $h' = h \times \text{id}_I$, and the corresponding projection π' , both quantifiers obey $h^*Q_\pi P = Q_{\pi'} h'^*P$.*

Proof. The first equivalence asserts that the first coordinate of every member of P lies in Q . The second asserts that each $x \in Q$ has every (x, a) in P . At $y \in Y$, either side of the reindexing equation quantifies $P(h(y), a)$ over exactly I . $\square$

Thus $\exists_\pi \dashv \pi^* \dashv \forall_\pi$. The subset hyperdoctrine organizes both operations [7, 8]: the existential projection used for ordinary completion has universal projection as its right-adjoint dual. In each Boolean fiber,

$$\forall_\pi P = \neg \exists_\pi \neg P. \quad (4)$$

At a semantic type index, existential quantification is the union of the instance predicates and universal quantification is their intersection. Both quantify over the same carrier with the same substitution operation.

For a bound U , let $\text{Sub}_{w, \rho}(U) = \{A \subseteq V \mid A \subseteq \llbracket U \rrbracket_{w, \rho}\}$. The universal type interpretation is

$$\llbracket \forall \alpha \leq U. T \rrbracket_{w, \rho} = \bigcap_{A \in \text{Sub}_{w, \rho}(U)} \llbracket T \rrbracket_{w, \rho[\alpha := A]}. \quad (5)$$

This intersection is an element of $\mathcal{U}$ and is available as another type instance. Quantification is therefore impredicative. Arbitrary placement of the constructor in (1) gives higher rank. Interpretation recurses on finite syntax over the already fixed set $\mathcal{U}$; operational inhabitants supply the realizers of the predicates [10].

Every bound admits the empty predicate. Hence

$$\llbracket \forall \alpha. \alpha \rrbracket = \emptyset, \quad \llbracket \forall \alpha. \text{List}(\alpha) \rrbracket = \{[]\}.$$

Instantiation at $\emptyset$ proves the exclusions; the empty list belongs to every list instance. More generally, $\exists f. \forall \alpha. P(f, \alpha)$ requires one uniform subject, whereas $\forall \alpha. \exists f. P(f, \alpha)$ permits a subject chosen separately at each instance.

2.5 Function types and application

For a computation e , write $e \in \text{Comp}(B)$ when every successful return of e belongs to B . For a completed value, $v \in \text{Val}(B)$ means $v \in B$. Constraint failure, written fail , is an unsuccessful computation outcome.

Definition 2.3 (Partial arrow). *For predicates $A, B \subseteq V$, define*

$$A \rightarrow B = \{f \in F \mid \forall p \in A. \forall v. f \, p \Downarrow v \implies v \in B\}. \quad (6)$$

A unary argument is identified with its unary packet.

An arrow is thus a universal behavioral predicate on one operational subject. Its result predicate is independent of the bound packet. In particular, $\emptyset \rightarrow B = F$, and a closure that always fails belongs to $A \rightarrow \emptyset$. The latter can contain closures even though $\text{Val}(\emptyset)$ is empty.

Proposition 2.4 (Arrow algebra). *For all predicates A, B, C, D ,*

$$\begin{aligned} A \subseteq C, \quad D \subseteq B &\implies (C \rightarrow D) \subseteq (A \rightarrow B), \\ (A \cup C) \rightarrow B &= (A \rightarrow B) \cap (C \rightarrow B), \\ A \rightarrow (B \cap D) &= (A \rightarrow B) \cap (A \rightarrow D). \end{aligned}$$

When α is independent of A , $A \rightarrow \llbracket \forall \alpha. T \rrbracket = \bigcap_{X \in \mathcal{U}} (A \rightarrow \llbracket T \rrbracket_{[\alpha := X]})$.

Proof. The first inclusion restricts the inputs tested and weakens the required result. The second splits the input quantifier over a union; the third requires both result predicates on each successful return. The final equation exchanges the universal quantifiers over inputs, returns, and type instances. $\square$

Input contravariance and result covariance follow from the interpretation. Conjoined arrows retain their correlations: $(\text{int} \rightarrow \text{string}) \wedge (\text{string} \rightarrow \text{int})$ specifies two input regions of the same closure. The predicate $\bigcap_{A \in \mathcal{U}} (A \rightarrow A)$ contains partial identities. Indeed, choosing $A = \{x\}$ forces every successful return on x to equal x .

An implementation constrained by a type has predicate $\{f\} \cap \llbracket T \rrbracket$. For closure and packet sets define

$$\text{App}(X, Q) = \{v \mid \exists f \in X. p \in Q. f \, p \Downarrow v\}.$$

Sharing remains in the joint environment until projection.

Proposition 2.5 (Application refinement). $\text{App}(X \cap Y, Q) \subseteq \text{App}(X, Q) \cap \text{App}(Y, Q)$.

Proof. A closure and packet witnessing the left side witness both right-hand memberships. The reverse inclusion can fail: distinct closures may return the same value on a packet while their singleton predicates are disjoint. $\square$

Lemma 2.6 (Type substitution). *For every well-formed, capture-avoiding substitution, including its action on reference graphs,*

$$\llbracket T[B/\alpha] \rrbracket_{w, \rho} = \llbracket T \rrbracket_{w, \rho[\alpha := \llbracket B \rrbracket_{w, \rho}]}$$

The instance B may itself contain universal quantifiers.

Proof. Induct on T . The variable case is substitution into ρ ; the reference case is Assumption 2.1. Structural and lattice constructors preserve the induction equations. At a fresh universal binder, the bound equation gives the same indexing set and the body equation gives equal predicates at every index. For arrows, substitution leaves the operational application relation fixed and substitutes its two predicates. $\square$

2.6 Quantifier algebra and refinement

For a common range I , and a predicate Q independent of a ,

$$(\forall a \in I.P) \wedge (\forall a \in I.R) = \forall a \in I.(P \wedge R), \quad (7)$$

$$(\exists a \in I.P) \wedge Q = \exists a \in I.(P \wedge Q), \quad (8)$$

$$(\forall a \in I.P) \wedge Q = \forall a \in I.(P \wedge Q) \quad (I \neq \emptyset). \quad (9)$$

Membership proves these identities. Type bounds always have nonempty ranges, so the last identity applies to independent fields under a type binder. Independent universals commute by exchanging their indices.

Bounds remain part of the predicate. In particular,

$$(\forall A \subseteq B.P(A)) \wedge (\forall A \subseteq C.P(A)) = \forall A \in \text{Sub}(B) \cup \text{Sub}(C).P(A).$$

This range can be smaller than $\text{Sub}(B \cup C)$: a mixed integer–string predicate lies below neither of those disjoint kinds. Shrinking a universal’s range weakens its obligation, while conjoining another predicate strengthens the constrained subject.

Universal projection preserves intersections. Its interaction with unions is strictly different. With range $\{0, 1\}$, take $P(a, z) \equiv z = a$ and $Q(a, z) \equiv z = 1 - a$. Then $\forall a.(P \vee Q) = \{0, 1\}$, while $(\forall a.P) \vee (\forall a.Q) = \emptyset$. Projection can also lose correlations: for $R_i = \{(s, i)\}$, first-coordinate projection p gives $p(R_0 \cap R_1) = \emptyset$ but $p(R_0) \cap p(R_1) = \{s\}$.

Theorem 2.7 (Monotone refinement). *Let $P' \subseteq P$ on a common scoped context. Any composite Q of reindexing and existential or universal projection over fixed ranges satisfies $Q(P') \subseteq Q(P)$.*

Proof. Inverse image preserves inclusion. An existential witness for P' is a witness for P ; satisfaction of all P' -instances implies satisfaction of all P -instances. Composition of these monotone maps preserves inclusion. $\square$

Live bounds fit this theorem by retaining their coordinates. A bounded universal is the fixed formula

$$\forall A \in \mathcal{U}. (A \subseteq \llbracket U \rrbracket_{w, \rho} \implies P(w, \rho, A)).$$

Refining the enclosing graph restricts w in this formula. An established empty observation remains empty; a singleton observation can retain its atom or become empty.

Theorem 2.8 (Data-relative conservativity). *Let $i : D \hookrightarrow V$ be the data embedding. Under Assumption 2.1, every legacy description T in a legacy environment η satisfies*

$$i^{-1}(\llbracket T \rrbracket_{i\eta}^+) = \llbracket T \rrbracket_{\eta}^0.$$

Using the base evaluation and observation rules on legacy syntax preserves legacy data observations.

Proof. Induct on legacy elaboration. The base predicates restrict to their original sets, and inverse image preserves intersection and union. Data constructors and copied references preserve the embedding and sharing. Local hiding uses the original data witnesses. The unchanged base evaluation and observation rules give the same data and preference selections by induction on their derivations. $\square$

Global top ranges over V , and its data restriction is the original top. The extension is consequently conservative at the data interface while providing additional operational subjects and predicates on them.

3 Certification of open descriptions

The semantic interpretation specifies which inhabitants satisfy an interface. Certification supplies finite evidence that a particular implementation does so. For an open implementation the statement is relative to its surrounding constraints, and its evidence retains the references on which the statement depends.

3.1 Stores, bounds, and proof support

A store $S \subseteq \mathcal{W}_E$ is a predicate on a fixed exposed interface. Write $S' \sqsubseteq S$ for $S' \subseteq S$, and $S \models \phi$ for $\forall w \in S. \phi(w)$. A context extension first pulls the store back along the map forgetting its fresh coordinates. Store comparisons are then made on the common context.

A rigid type context is an ordered list $\Delta = \alpha_1 \leq U_1, \dots, \alpha_n \leq U_n$, with each bound using earlier variables only. Write $\rho \models_w \Delta$ for satisfaction of these inclusions at w . A fact certificate

$$S; \Delta \vdash \phi$$

proves $\forall w \in S \forall \rho \models_w \Delta. \phi(w, \rho)$. A certificate is a finite proof graph checked by a kernel. Its leaves are kernel axioms, independently available source facts, or explicitly named hypotheses; its inference dependencies are well founded. An open hypothesis remains displayed until a rule discharges it or linking supplies its evidence.

A goal has its own assumption store. The full source relation contains the interface being required; a proof of that interface uses the independently justified premises recorded in its support. In particular, an implementation's result annotation is a proof goal. Using a callback's contract under a named parameter hypothesis is justified by later certified substitution. These two roles of a contract are determined by its proof scope.

For a reference r , upper and lower bounds are certificates

$$S; \Delta \vdash \tau_r \preceq U_r, \quad S; \Delta \vdash L_r \preceq \tau_r, \quad (10)$$

where $\preceq$ in a certificate denotes pointwise semantic inclusion. Intersecting established upper bounds and joining established lower bounds improves precision. Relational facts among several references remain available alongside these summaries. For closed summaries, the optimal bounds are

$$U_S^*(r) = \bigcup_{\substack{w \in S \\ \rho \models_w \Delta}} \llbracket \tau_r \rrbracket_{w, \rho}, \quad L_S^*(r) = \bigcap_{\substack{w \in S \\ \rho \models_w \Delta}} \llbracket \tau_r \rrbracket_{w, \rho}.$$

A computed pair is sound when $L_r \subseteq L_S^*(r)$ and $U_S^*(r) \subseteq U_r$. These equations characterize precision; the checker manipulates represented bounds and their proofs. Empty unions and intersections have values $\emptyset$ and V , respectively. An independently refuted store is reported as inconsistent.

Lemma 3.1 (Persistence). *If $S; \Delta \vdash \phi$ and $S' \sqsubseteq S$, then the same proof with a weakened store premise establishes $S'; \Delta \vdash \phi$. Moreover, $U_{S'}^*(r) \subseteq U_S^*(r)$ and $L_S^*(r) \subseteq L_{S'}^*(r)$.*

Proof. Every world and rigid valuation admitted by the stronger store is admitted by the original one. Restricting the indices of a universal assertion preserves it; restricting a union shrinks it and restricting an intersection enlarges it. $\square$

The target expression, scope, and rebinding map are fixed in this lemma. A proved upper bound on X licenses its use in the proved direction. For example, $X \preceq \text{int}$ permits an X -argument to be used arithmetically; membership of a particular integer in X requires the opposite-direction fact. In an arrow domain the inclusion reverses: $\text{int} \rightarrow B \preceq X \rightarrow B$. Each approximation of a compound expression therefore carries a proof for that expression.

The *support* $\text{supp}(q)$ of a judgment is the transitive closure of its free graph references, lexical binders, proof hypotheses, and protocol dependencies. Substitution, freshness checks, and proof-cache keys use this support. A cached theorem is reusable by persistence. A result conditional on a temporary equality retains that equality as a premise, and a search status is revisited when its dependencies change.

3.2 Values, computations, and elimination evidence

Let $\Gamma = x_1 : T_1, \dots, x_n : T_n$ record hypotheses about values available on successful completion. The two core judgments are

$$S; \Delta; \Gamma \vdash_v v : T, \quad S; \Delta; \Gamma \vdash e : \text{Comp}(T).$$

A source data reference contributes its established bound together with a *validation demand*: the operand must satisfy all its retained source constraints when demanded. A function reference contributes invocation evidence or an explicit hypothesis to be discharged at linking. This distinction allows a body to be checked using a partial capture while retaining the work required to obtain its value.

Description lookup is distinct from materialization. A required declaration $z! : \text{int}$ establishes an integer bound in every successful completion without supplying a value for z . Both a lexical reference and a field selection may use that bound in a body proof. Neither operation discharges the runtime presence and validation demands. An optional declaration alone does not establish that its field is available.

The core contains variables, literals, closures, strict application and let, records, tuples, selection, Boolean conditionals, meet, assertions, primitives, and fail. It executes completed subjects of the source graph. In particular, source unification of open records shares a common completion before the core performs its completed-value check. The source checker therefore collects the declarations of a record meet before checking its fields. Internal references are rebound to that common subject; external references retain their lexical coordinates. Named descriptions, embeddings, and abbreviations preserve this rule. Collection retains required and optional presence, definition boundaries, and source observations. A description's current fields do not by themselves establish an exact inventory. In particular, checking a method before collecting its sibling refinements would demand a theorem about an operand that the source never executes alone.

Assumption 3.2 (Primitive and completion interface). *A primitive supplies a packet interface, a successful-result interface, and finite transport of their evidence. On complete certified operands it returns a value, fails, or takes a primitive step. Every such step preserves the result certificate. Source completion and core execution simulate one another on successful data observations, retaining the source's packet and validation demands. Failure retains its branch and scope.*

For a registered native builtin, its implementation-owned descriptor supplies this primitive evidence. Its parameter constraints establish operand coverage; its result kind bounds every successful return. More precise native signatures retain container elements and converted record fields, including optional presence. This evidence does not add runtime constraints that could materialize an incomplete operand. Frozen package declarations retain their lexical scopes. Package labels and defaults belong to this protocol. Attaching a client arrow creates a separate proof goal and cannot strengthen the implementation's evidence by assertion. A foreign function without such a primitive contract requires independent linking evidence. Validator constructors check their saved arguments and describe inhabitants of the validated slot's type.

Some primitives admit more precise structural rules. The conjunction of a nonempty list of A -values has result A , and a disjunction of A -values has result A on every successful return. A conjunction's optional tail cannot supply a mandatory result constraint. In particular, $\text{and}([])$ is $\top$, so the unconstrained finite-list domain does not justify the universal arrow $\forall A. \text{List}(A) \rightarrow A$ for and . The nonempty-list domain does. The empty disjunction has no successful result. Length retains proved bounds; closing a record preserves its input constraints. A rigid predicate is proof data, and executing it as a concrete record must not manufacture a failure certificate.

Operator rules likewise establish operand coverage before deriving a result. Arithmetic, ordered comparison, Boolean connectives, regex matching, interpolation, and equality use their enabled base-language domains. Each alternative must support the operation. Integer list indexing and slicing are partial computations that retain the possible element predicates; record selection requires field evidence. Open structural descriptions cannot be executed as completed singleton values to prove an equality result.

Proof decorations may record type abstraction $\Lambda\alpha.e$ and instantiation $e[T]$. Their *erasure* removes the decorations: $|\Lambda\alpha.e| = |e|$ and $|e[T]| = |e|$. It retains code, captures, protocols, defaults, and executable assertions. Thus $|e \ \& \ P| = |e| \ \& \ P$ for an assertion.

The main rules are as follows. A missing type-context prefix in a premise is the common $S; \Delta$.

$$\begin{array}{c}
\frac{x : T \in \Gamma}{S; \Delta; \Gamma \vdash_v x : T} \text{VAR} \quad \frac{S; \Delta; \Gamma \vdash_v v : T}{S; \Delta; \Gamma \vdash v : \text{Comp}(T)} \text{RETURN} \quad \frac{S; \Delta; \Gamma, x : A \vdash e : \text{Comp}(B)}{S; \Delta; \Gamma \vdash_v \lambda x. e : A \rightarrow B} \text{ABS} \\
\\
\frac{S; \Delta; \Gamma \vdash f : \text{Comp}(A \rightarrow B) \quad S; \Delta; \Gamma \vdash a : \text{Comp}(A)}{S; \Delta; \Gamma \vdash f a : \text{Comp}(B)} \text{APP} \\
\\
\frac{S; \Delta; \Gamma \vdash e : \text{Comp}(A) \quad S; \Delta; \Gamma, x : A \vdash d : \text{Comp}(B)}{S; \Delta; \Gamma \vdash \text{let } x = e \text{ in } d : \text{Comp}(B)} \text{LET} \quad \frac{}{S; \Delta; \Gamma \vdash \text{fail} : \text{Comp}(\perp)} \text{FAIL} \\
\\
\frac{S; \Delta; \Gamma \vdash e : \text{Comp}(A) \quad S; \Delta; \Gamma \vdash d : \text{Comp}(B)}{S; \Delta; \Gamma \vdash e \wedge d : \text{Comp}(A \wedge B)} \text{MEET} \quad \frac{S; \Delta; \Gamma \vdash e : \text{Comp}(A) \quad S; \Delta \vdash A \preceq B}{S; \Delta; \Gamma \vdash e : \text{Comp}(B)} \text{SUB} .
\end{array}$$

Value subsumption has the corresponding form. Literals synthesize singletons. A record or tuple constructor combines its component certificates; selection requires a guaranteed field or a certified tuple coordinate. General list indexing is a checked primitive. Conditionals require a Boolean condition and certificates for both branches. A branch may use facts implied by its checked condition, scoped to that branch. Primitive application uses the packet and result evidence in Assumption 3.2.

Both judgments admit intersection introduction for the same erased term and union introduction. Write $\vdash_\star e : T$ for either judgment, with T inside Comp in the computation case. Their universal rules are

$$\frac{S; \Delta, \alpha \leq U; \Gamma \vdash_\star e : T \quad \alpha \notin \text{supp}(S, \Gamma, |e|)}{S; \Delta; \Gamma \vdash_\star e : \forall \alpha \leq U. T} \text{ALL-I}$$

$$\frac{S; \Delta; \Gamma \vdash_\star e : \forall \alpha \leq U. T \quad S; \Delta \vdash B : \text{Type} \quad S; \Delta \vdash B \preceq U}{S; \Delta; \Gamma \vdash_\star e : T[B/\alpha]} \text{ALL-E}.$$

The variable α is fresh for Δ, U . An instance is a finite scoped type expression, including a quantified expression or a live reference. Generalization retains one implementation independently of the rigid instance; elimination records the chosen expression and its bound proof.

Union elimination uses one evaluated subject:

$$\frac{S; \Delta; \Gamma \vdash_v v : A \vee B \quad S; \Delta; \Gamma, x : A \vdash d : \text{Comp}(C) \quad S; \Delta; \Gamma, x : B \vdash d : \text{Comp}(C)}{S; \Delta; \Gamma \vdash d[v/x] : \text{Comp}(C)}. \quad (11)$$

Here x is fresh for Γ, v ; the value version is analogous. A general scrutinee is first named by strict let. Both branches certify the same continuation. Thus the proof preserves the evaluated subject and its sharing, as in value-restricted contextual union checking [19].

A *checkable predicate* P is a represented first-order constraint whose forwarding test has the primitive contract: a successful test returns its operand in P , and a failed test produces `fail`. Live operands retain their validation demands. Its rule is

$$\frac{S; \Delta; \Gamma \vdash e : \text{Comp}(A) \quad S; \Delta \vdash P \text{ checkable}}{S; \Delta; \Gamma \vdash e \& P : \text{Comp}(A \wedge P)}. \quad (12)$$

The assertion is executable. Its successful postcondition supplies the narrower certificate; further conjuncts remain attached to the demanded predicate.

3.3 Subtyping and implementation attachment

Subtyping evidence includes reflexivity, explicit transitivity, bottom and top, base-kind inclusions, literal membership, distributive-lattice rules, record width/depth and presence, and covariant tuples and lists. Bound lookup gives $\alpha \preceq U$ for $\alpha \leq U \in \Delta$. Proved store facts supply reference inclusions. The function and universal rules are

$$\frac{S; \Delta \vdash A \preceq C \quad S; \Delta \vdash D \preceq B}{S; \Delta \vdash (C \rightarrow D) \preceq (A \rightarrow B)} \quad \frac{S; \Delta \vdash B \preceq U}{S; \Delta \vdash (\forall \alpha \leq U. T) \preceq T[B/\alpha]} \quad \frac{S; \Delta, \alpha \leq U \vdash C \preceq T \quad \alpha \notin \text{supp}(S, \Delta, C)}{S; \Delta \vdash C \preceq (\forall \alpha \leq U. T)}.$$

Proposition 2.4 supplies the arrow equalities. Each subtyping rule transports value evidence: arrows retain an invocation derivation, records retain component evidence, and unions retain arm evidence. This transport is part of a core subsumption certificate. Purely semantic inclusion facts may also be retained for reasoning about predicates.

An implementation comprises its origin, captures, body e , and declared packet interface C . Attaching a requirement $A \rightarrow B$ creates the goals

$$S; \Delta \vdash A \preceq C, \quad S; \Delta; \Gamma, x : A \vdash e : \text{Comp}(B). \quad (13)$$

The first certifies the calling capability; the second certifies the body under exactly the promised input and its certified captures. All accumulated requirements are retained, together with their body derivations. A result annotation supplies the second goal; an assertion supplies a test governed by (12).

For a live input X bounded by integers, the arithmetic primitive yields

$$S \vdash X \preceq \text{int} \implies S; \emptyset; x : X \vdash 2 * x : \text{Comp}(\text{int}).$$

The implementation consequently has a certificate at $X \rightarrow \text{int}$ using only that bound. Returning x similarly establishes result X . Returning the constant 2 at result X requires $\{2\} \preceq X$ or a retained assertion that establishes membership on success. The input and result directions are therefore determined by their proof obligations.

An invocation creates fresh packet cells and links a result cell to the body. For the argument 2 and the preceding arithmetic body, its source constraints include

$$p : \tau_r, \quad p : 2, \quad a = 2p, \quad a : \text{int}. \quad (14)$$

The packet references the live description according to its copy/rebinding map; it does not write its argument back into the declaration. Later constraints on the packet or result conjoin with this activation. Deriving $a = 4$ preserves the packet's validation demand, including when a body ignores an operand. For a higher-order argument, linking also discharges the coverage goal in (13). Narrowing a call packet and proving a universal capability are distinct judgments.

3.4 Relevant descriptors and explicit observations

A partial arrow constrains successful returns. A source interface additionally records which observations it explicitly claims. A *relevant descriptor* is a pair $\mathcal{R} = (T, \Omega)$, where T is its semantic type and Ω a finite graph of explicit observations. The graph retains roots, arrow clauses, type binders, alternatives, field-presence guards, and dependencies. Its semantic erasure is T .

For a conjunctive arrow root $C = \bigwedge_{i \in I} (A_i \rightarrow B_i)$, define

$$D_C = \bigcup_i \llbracket A_i \rrbracket, \quad \mathcal{B}_C(p) = \bigcap_{i: p \in \llbracket A_i \rrbracket} \llbracket B_i \rrbracket \quad (p \in D_C).$$

For each nonempty $J \subseteq I$, its exact input region and combined result are

$$R_J = \left(\bigcap_{j \in J} \llbracket A_j \rrbracket \right) \setminus \left(\bigcup_{i \notin J} \llbracket A_i \rrbracket \right), \quad B_J = \bigcap_{j \in J} \llbracket B_j \rrbracket. \quad (15)$$

They are indexed by w, ρ . The inhabited R_J partition the stated domain, and B_J is the obligation on its region. Complement is an internal operation on these guards.

Observation traversal collects every conjunct of an explicit subject, descends through written components and both arrow positions, and keeps alternatives under their guards. Universals are examined under rigid variables. An independent sibling domain may also determine a finite structural instance of a universal clause: matching $\alpha \rightarrow \alpha$ against a sibling integer domain anchors the instance $\text{int} \rightarrow \text{int}$. Repeated variables must match identically, and the substitution and bound proofs must check. The dependency graph records the sibling's independence through lexical transformations.

These are source observations. An inferred intermediate type or a call instance retains the root of its declaration. Writing an instance as a separate interface introduces a root for that interface. In particular, a generic result $\alpha \wedge \beta$ is examined under rigid α, β , while an explicit result $\text{int} \wedge \text{bool}$ has a ground contradiction. Observation origins remain through semantic absorption and simplification.

For a root in rigid context Δ , define world predicates

$$\begin{aligned} \text{Void}_\Delta(B)(w) &\iff \forall \rho \models_w \Delta. B(w, \rho) = \emptyset, \\ \text{Empty}_\Delta(R)(w) &\iff \forall \rho \models_w \Delta. R(w, \rho) = \emptyset, \\ \text{Reach}_\Delta(R)(w) &\iff \exists \rho \models_w \Delta \exists p. p \in R(w, \rho). \end{aligned}$$

A checked proof of $\text{Void}_\Delta(B_J)$ creates a demand that R_J be empty. An enclosing alternative or presence guard restricts the store of that demand. The outcomes are determined by positive evidence:

Outcome	Required evidence in the retained scope
Discharged	$S \models \text{Empty}_\Delta(R_J)$.
Rejected	$S \models \text{Void}_\Delta(B_J)$ and $S \models \text{Reach}_\Delta(R_J)$.
Pending	The result has a uniform emptiness proof; the input question remains unresolved.

A completed mandatory pass is *quiet* at a root when it has discovered no uniform result refutation. Quietness records completed search, and supplies no logical premise. New bounds or evidence re-enqueue the root. A guarded rejection becomes global only when its guard is required or all remaining alternatives are rejected. Semantic emptiness of a guard discharges its local demands; relevance rejection itself supplies no semantic emptiness fact.

Definition 3.3 (Current relevant admission). *For a finite checking kernel K , $\text{RC}_K(S, \Delta, \Omega)$ holds when its mandatory traversal is complete and every required root is discharged or quiet. The admission record retains the observation graph, proof support, and triggers for renewed checking.*

Thus $\text{int} \rightarrow \perp$ is rejected as an explicit interface even though it has failure-only realizers. The generic result $\alpha \wedge \beta$ has no uniform emptiness proof: $\alpha = \beta = \{0\}$ gives an inhabited instance. The distinction is between a semantic type and its stated observations.

Proposition 3.4 (Persistent rejection and observation coherence). *A checked rejection persists under store refinement in its retained scope. Permutation, reassociation, duplication, alpha-renaming, and replacing $A \rightarrow (B \wedge C)$ by $(A \rightarrow B) \wedge (A \rightarrow C)$ preserve observation questions when dependencies and completed kernel work agree.*

Proof. The rejection premises are positive facts about every admitted world; Lemma 3.1 preserves both. The listed transformations preserve $\mathcal{B}_C(p)$ at every stated packet. Result splitting duplicates the input guard and conjoins the same results. Canonical duplicate elimination and scope-preserving renaming consequently identify the region and anchor graphs. $\square$

3.5 Refuting an open goal

Refutation of an open implementation must also respect its admitted worlds. A sufficient successful-result counterexample for $f : A \rightarrow B$ is

$$\forall w \in S. \exists \rho \models_w \Delta, p, v. \quad p \in \llbracket A \rrbracket_{w, \rho} \wedge f_w p \Downarrow v \wedge v \notin \llbracket B \rrbracket_{w, \rho}. \quad (16)$$

The kernel verifies the execution and membership evidence. A witness may vary with w ; a fixed ground witness is a useful special case. At a closed universal interface one admissible instance suffices. An analogous certificate exhibiting a packet in $A \setminus C$ in every world refutes the coverage requirement of (13). Coverage and relevant-admission refutations concern their respective disciplines; (16) refutes the semantic contract.

A point in an upper approximation is only a possible witness. A live domain bounded by number can later become an integer domain, so a fractional point in that upper approximation alone establishes no persistent violation. A fixed numeric domain with an identity body and integer result does have the persistent witness 1.5. Failure and divergence supply no successful-result counterexample to a partial arrow. Failed proof search leaves its goal pending.

4 Evaluation by certified propagation

Certification consumes facts about an open graph; evaluation can produce those facts. Both processes therefore operate on the same retained source relation. Their common invariant is exact residualization: each derived result carries the constraints under which it was obtained.

4.1 Exact residuals

Let G be a scoped graph and P_G its predicate of successful joint completions on an exposed interface E . The interface includes every coordinate available to later source unification. After hiding only fresh implementation-local nodes, a valid normalization $G \rightsquigarrow G'$ satisfies

$$P_{G'} = P_G \text{ on } E, \quad P_{G'} \wedge R = P_G \wedge R \text{ for every future constraint } R \text{ on } E. \quad (17)$$

The second equality follows from the first on this common interface. Its force is the choice of interface: the retained graph includes live descriptions, reference maps, binder scopes, closure origins, defaults, and validation dependencies. It also retains the certification goals and source observations needed to reconstruct the checking problem. Scalar projection alone loses information needed by future source constraints.

A normalization can be justified conditionally. A *guard* H is a retained predicate restricting the worlds in which its proof applies.

Proposition 4.1 (Guarded normalization). *If $P = P'$ under $S \wedge H$, then*

$$S \wedge P = S \wedge ((H \wedge P') \vee (\neg H \wedge P)).$$

Proof. At a world in S , the H branch uses the assumed equivalence and the complementary branch retains P . This proves equality at every world. $\square$

A cache can represent the guarded result together with the original graph, sharing the complementary residual. Its hypotheses remain part of its support. Establishing a theorem under a proposed approximation therefore records a conditional theorem; using it unconditionally requires a proof of that approximation.

For the activation in (14), arithmetic may establish $a = 4$. The residual retains $p : \tau_r$ and the validation connecting p to the argument 2. A later constraint excluding 2 from τ_r still refutes the call, and a later result constraint excluding 4 still refutes its result. The same requirement preserves demanded arguments of constant functions.

4.2 States and transitions

An evaluator state has the form

$$N = (G, \mathcal{F}, \mathcal{C}, \mathcal{O}, W).$$

Here G is the exact graph; $\mathcal{F}$ contains proved facts and bounds; $\mathcal{C}$ contains core, inclusion, and protocol certificates; $\mathcal{O}$ contains goals and observation triggers; and W is the worklist. Facts and certificates carry scope and proof support. Constraints accumulate by conjunction, evidence by set union. Scheduling data have no semantic force.

The graph contains the semantic predicates of attached interfaces. A proof uses its independently justified assumption store, as defined in Section 3. This separates conjunction of requirements from establishment of the evidence needed to invoke an implementation.

The transition rules have five forms. *Refinement* conjoins source information and installs its scoped goals. *Propagation* checks and records a consequence of established premises. *Certification* verifies a finite derivation and its remaining hypotheses. *Reduction* performs a justified primitive or function step, preserving (17) and all operand demands. Finally, *refutation or suspension* records either checked negative evidence or the unresolved obligation. Every new fact re-enqueues the nodes depending on it.

An invocation is enabled by the required grounded core and protocol evidence and a completed current admission pass. A symbolic reduction may expose useful results while data demands remain. A *derived atom* is such a summary with its guards and validation dependencies. A *completed value* has discharged its current demanded validations and required certification obligations. Export preserves its live source dependencies for subsequent refinement.

```
propagate(state, budget):
    while work_remains(state) and budget_allows_step(budget):
        node = take_work(state)
        for item in one_accounted_step(node, state):
            verify_derivation_or_exact_rewrite(item)
            if accumulate_with_scope_and_support(state, item):
                enqueue_dependents(state, item)
        return retained_residual(state), scoped_diagnostics(state)
```

The dependency graph permits the cycle of useful work

evaluation $\longrightarrow$ stronger bounds $\longrightarrow$ certification $\longrightarrow$ evaluation.

Accumulating partial information and notifying dependents follows the propagator model [22]; the bounds are abstract summaries of the retained concrete relation [23].

4.3 Grounding, suspension, and renewed checking

Evidence dependencies are finite proof dependencies. A callable hypothesis used to check a client is discharged by the actual implementation’s certificate. An implementation’s pending annotation, and consequences borrowed from that annotation, remain excluded from its own independent premise set. A cycle of unproved goals therefore remains a cycle of residual obligations. An independent data bound can establish a body theorem while the computation producing that data remains cyclic or incomplete. Grounding a proof and completing its data are separate conditions.

A residual obligation records its target, scope, dependencies, and reason for waiting: missing data, missing evidence, a dependency cycle, or unfinished budgeted work. Conditional reductions retain their hypotheses. A finite checked inference may discharge hypotheses, including a supplied inductive proof; membership in a dependency cycle supplies no such inference.

Under further source conjunction, existing theorems about fixed references persist by Lemma 3.1. Additional requirements generate additional goals. Current relevant admission and snapshot-dependent search results are rechecked against their full dependency support. In this sense recertification is the continued production of evidence for the live graph, with theorem reuse justified by its explicit premises.

4.4 The mandatory checking kernel

A conforming kernel verifies the core rules, substitutions, instance bounds, proof support, and protocol evidence. Its finite mandatory traversal includes these direct checks on represented inputs:

1. Base-kind disjointness, incompatible literals, finite literal unions, and constant numeric intervals, including integer rounding at strict bounds; ground tests use the enabled base primitives.
2. Required-field and presence conflicts, excluded exact-record fields, and fixed tuple lengths and components. Impossible optional contents imply absence, with the appropriate presence guard.
3. Ground primitive steps, invalid known indices, malformed argument order, duplicate slot bindings, unknown labels, and missing required slots.
4. Guarded Boolean normalization. A union is semantically empty when every alternative is refuted. A universal is empty when a checked admissible instance is empty; instantiating $\forall \alpha. \alpha$ at $\perp$ is a direct case.
5. Aggregation of equal-domain arrow clauses, exact-region checks, and verified structural anchors. The result check includes every clause active in a region. Refuted results with unresolved input reachability create pending demands.

Ground, interval, record, and tuple witnesses supply direct reachability certificates when their memberships hold throughout the current store. For instance, $\{0, 1\}$, $\{1, 2\}$, and $\{0, 2\}$ have inhabited pairwise intersections and an empty triple intersection. Their three arrows over the same fixed integer domain are jointly rejected by the mandatory region check. Boolean decision graphs may share regions and proof subterms. Unfinished mandatory work remains pending.

Additional proof search may be arbitrary; its accepted consequences are checked by the kernel or by an extension satisfying the same soundness and evidence transport conditions. A proved refutation, an unanswered goal, an inconclusive search, and resource exhaustion are distinct outcomes. Only the first supplies negative evidence.

4.5 Work bounds and export

Accounted work includes graph expansion, copying, matching, substitution, normalization, integer bit operations, and proof verification. For an accounted proof graph of L nodes, each with local checking cost at most C , verification uses $O(LC)$ time and $O(L)$ graph space, in addition to explicitly represented payloads. Region generation and large instances are charged when constructed. A fixed work allowance bounds the performed work and yields a residual at the end of any finite run.

Proposition 4.2 (Finite-basis saturation). *On a fixed finite fact-and-obligation basis, fair iteration of finitely many monotone inflationary propagators reaches the least common fixed point above its initial information. This fixed point is independent of scheduling. Every finite prefix is sound when its local proof obligations are checked.*

Proof. Only finitely many strict additions are possible. Fairness applies each propagator after the last addition, so the resulting set is a common fixed point. Monotonicity keeps every intermediate set below any common fixed point above the initial one, proving leastness. Induction over checked additions proves prefix soundness. $\square$

Evaluation may generate new syntax and obligations. Equation (17) justifies every bounded prefix independently of saturation. Quiet admission statuses are observations of completed checking work, rather than logical facts in the propagation basis.

An exported source residual preserves the exposed interface, scope and origin metadata, reference graph, validation demands, and observation roots. Proof objects may be retained or reconstructed from their premises. Re-import first reinstalls the obligations, merges additional constraints, and schedules their affected nodes. A default-selected presentation retains the original alternatives when intended for subsequent unification; comprehension expansion retains its base observation dependencies. Thus source export preserves the refinement problem as well as the currently displayed data.

5 Soundness and compositionality

The proof has two levels. Core certificates establish semantic membership and safe elimination at each completed world. State invariants then lift those results to partial evaluation of an open graph. The assumptions are the scoped base interpretation in Assumption 2.1, the primitive and completion interface in Assumption 3.2, and exact symbolic rewrites as in (17).

5.1 Substitution and semantic soundness

Lemma 5.1 (Certificate substitution). *A well-formed type substitution with evidence for its bound preserves core certificates, including their stores and reference maps. If $S; \Delta; \Gamma, x : A \vdash e : \text{Comp}(B)$ and $S; \Delta; \Gamma \vdash_v v : A$, then $S; \Delta; \Gamma \vdash e[v/x] : \text{Comp}(B)$. Value and simultaneous substitution have the corresponding properties. Store refinement preserves each certificate for its fixed target and scope.*

Proof. Induct on the finite proof graph. Variable leaves use the supplied evidence; fact leaves use their substituted proofs. Structural rules rebuild from the induction hypotheses. Freshly rename a universal binder away from the substitution's support; its generalization condition is then preserved. At elimination use Lemma 2.6 and composition of capture-avoiding substitutions. Transform subtyping and transport proofs by the same induction. For union elimination, rename its local variable y and use $(d[u/y])[v/x] = (d[v/x])[u[v/x]/y]$. Each discharged hypothesis is replaced by its supplied derivation. Store weakening uses Lemma 3.1 at every fact leaf. The source observation roots keep their declaration scopes throughout this substitution. $\square$

Theorem 5.2 (Open certificate soundness). *Let $w \in S$, $\rho \models_w \Delta$, and let γ supply values satisfying Γ . If $S; \Delta; \Gamma \vdash_v v : T$, its interpretation under w, ρ, γ belongs to $\llbracket T \rrbracket_{w, \rho}$. If $S; \Delta; \Gamma \vdash e : \text{Comp}(T)$, every successful return under that interpretation belongs to $\llbracket T \rrbracket_{w, \rho}$. A conditional certificate has these conclusions whenever its displayed hypotheses hold.*

Proof. Induct on the certificate. Variables use the environment; literals use singleton membership. Constructors and selections use their component predicates. Abstraction extends γ by an arbitrary member of the input predicate; the body induction hypothesis proves the implication in (6). Application first obtains an input member and then uses that implication. Strict let substitutes its successful bound value. Failure has no successful return. Meet restricts a shared completion to both predicates, and assertions forward only passing operands.

Intersection introduction concerns the same execution. Union elimination chooses the actual value's arm and uses the common continuation. Subtyping rules establish inclusion, and primitives use Assumption 3.2. For universal introduction, its support condition permits varying the rigid valuation without changing the store, environment, or erased computation. Universal elimination uses Lemma 2.6. Fact leaves are valid under their recorded premises, and well-founded proof dependencies close the induction. $\square$

Definition 5.3 (Relevant certification). *A current relevant certificate for (T, Ω) comprises a core certificate at T , grounded attachment and protocol evidence, and $\text{RC}_K(S, \Delta, \Omega)$. It retains their supports and observation triggers. For $w \in S$, let $\text{Certified}_{S, K}(T, \Omega; w)$ be the completed values carrying this evidence with their free hypotheses discharged.*

Corollary 5.4 (Static refinement). *For every admissible rigid valuation,*

$$\text{Certified}_{S, K}(T, \Omega; w) \subseteq \llbracket T \rrbracket_{w, \rho}.$$

This inclusion can be strict.

Proof. Erase the admission record and apply Theorem 5.2 to the core certificate. A failure-only closure inhabits $\text{int} \rightarrow \perp$, whose explicit relevant root is rejected, witnessing strictness. $\square$

Relevant certification refines semantic types as a source discipline. Its admission component establishes neither termination nor a successful call.

5.2 Safety of the completed core

At a fixed world, substitute complete certified captures and operands. The core uses left-to-right strict evaluation contexts for application, let, constructors, selection, conditions, and primitive operands. Closure bodies execute when invoked. Principal reductions include

$$\begin{aligned} (\lambda x. e) v &\longrightarrow e[v/x], & \text{let } x = v \text{ in } e &\longrightarrow e[v/x], \\ \{\dots, \ell = v, \dots\}. \ell &\longrightarrow v, & [v_0, \dots, v_{n-1}][i] &\longrightarrow v_i \quad (0 \leq i < n). \end{aligned}$$

A Boolean condition selects its branch. A completed assertion returns its operand or fails. Completed meet returns the common subject when both constraints hold, and otherwise fails. Source open-record unification has already established the shared completion. Failure propagates through every strict context, including a demanded argument unused by its function body.

Lemma 5.5 (Canonical evidence). *A closed certified arrow value has closure and invocation evidence; a certified guaranteed field is present; a fixed tuple has the certified length; and a Boolean value is a Boolean literal. A nonfailure principal redex at a certified elimination has a reduct with the same result certificate.*

Proof. Induct on value evidence, including its transport derivations. Introductions give their shapes. Arrow transport restricts the input premise and transports the result. Intersection evidence retains both proofs: the common-domain arrow law intersects their results, and the common-result law checks the same body under both arms of a union input. Union evidence selects the value's actual arm; quantified evidence substitutes its recorded instance. The resulting invocation derivation justifies beta reduction by Lemma 5.1. Field, tuple, and conditional reductions use their component or branch proofs. Primitive reductions use Assumption 3.2. At an inhabited world, Theorem 5.2 excludes a certified value of bottom. $\square$

Theorem 5.6 (Core preservation and progress). *At a world satisfying the store and after certified substitution for free values, reduction preserves a core computation certificate. A certified computation is a value, is fail, or has a reduction step.*

Proof. Canonical evidence and substitution preserve nonfailure principal reductions. A failure reduct has certificate $\text{Comp}(\perp)$ and subsumes to the original result. Induction on strict contexts rebuilds the enclosing proof. Typing-only intersection and universal rules rebuild for the same reduct; a value-restricted union cut uses its selected arm evidence.

For progress, inspect the next demanded subexpression. Its step lifts and its failure propagates. When it is a value, canonical evidence supplies the required elimination shape, and the primitive contract supplies its next action. Closures are values. Structural induction exhausts the core forms. $\square$

5.3 Preservation and progress of open states

A state is *well evidenced* when every fact and certificate has a valid proof with its recorded support, every enabled invocation has its required evidence, and each simplification retains exact source demands and obligations.

Theorem 5.7 (State preservation). *The transitions of Section 4 preserve well-evidencedness. Propagation, certification, and valid normalization preserve P_G . Adding an external constraint R gives $P_G \wedge R$ on the common interface and preserves existing facts about fixed targets.*

Proof. Source refinement restricts the store and installs goals; persistence preserves the existing premises. Propagation and certification check each added proof with its support. Completed reductions use core preservation; symbolic reductions use (17). Refutation retains its evidence and scope, and suspension changes no predicate. Induction over transitions proves the invariant. $\square$

Theorem 5.8 (Progress with residual obligations). *For each demanded result, a well-evidenced state exposes a completed value, a justified scoped failure or rejection, an enabled transition, or an exact residual recording its unresolved obligations. Once an elimination's demanded operands and required evidence are complete, it is not stuck for lack of a supported elimination or callable protocol.*

Proof. Inspect the finite demand graph. A completed node has its validation and certification evidence; a refuted node has its negative proof. At a nonterminal node, every unavailable premise is recorded as a dependency, including a pending strongly connected component. If all premises are present, propagation, certification, or reduction is enabled. Core progress supplies the step for a completed elimination. A budget suspension retains the enabled work in the residual. These cases exhaust the graph's node forms. $\square$

The residual alternative specifies the meaning of partial evaluation. The core theorem supplies the stronger elimination guarantee when those recorded premises are available.

5.4 Composition and successful execution

At a world w , a closure is *failure-free on A* when its complete executions on A -packets never reach fail; divergence is allowed. It is *successful on A* when every such execution terminates in a value. These properties include the validation and adapter operations of its specified call protocol.

Theorem 5.9 (Open compositionality). *Suppose f and g have core and protocol certificates at $A \rightarrow B$ and $C \rightarrow D$ over $S; \Delta; \Gamma$, with $S; \Delta \vdash B \preceq C$. Then $h = \lambda x. g(f(x))$ has a core certificate at $A \rightarrow D$. For every $S' \sqsubseteq S$, $w \in S'$, admissible rigid valuation, and certified linking substitution, failure-freedom of f on $\llbracket A \rrbracket$ and of g on $\llbracket C \rrbracket$ implies failure-freedom of h on $\llbracket A \rrbracket$. Success of both components on those domains implies success of h , with result in $\llbracket D \rrbracket$. These claims are preserved by every well-formed type substitution, including impredicative instances.*

Proof. Under $x : A$, application gives $f(x) : \text{Comp}(B)$, subsumption gives $\text{Comp}(C)$, and another application gives $g(f(x)) : \text{Comp}(D)$. Abstraction establishes the composite type. Persistence and certified substitution transport the proof to the chosen store, world, and linking.

Strict execution first runs $f(x)$. A diverging first computation admits no later failure. A successful return y satisfies $\llbracket B \rrbracket \subseteq \llbracket C \rrbracket$ by soundness, so the premise on g applies. For successful components the two finite executions concatenate and the result satisfies $\llbracket D \rrbracket$. Type substitution preserves the proof and its erased execution. $\square$

A further caller-side packet or result constraint contributes a validation demand. The corresponding failure-free claim includes that demand in its premises. A stronger published interface similarly contributes an additional attachment goal and observation root; the original theorem retains its original target.

5.5 Coherence and monotone observations

Proof erasure preserves the source implementation, its captures, protocol, defaults, and assertions. The exact source graph persists independently of whether proof objects are retained in executable code.

Theorem 5.10 (Erasure and declaration coherence). *At a fixed admitted world, grounded certificates for the same implementation origin and source graph give the same erased executions. Reassociation, permutation, duplication, and common-domain result splitting of interface conjuncts preserve the exact predicate, body, and observation questions.*

Proof. Evaluation acts on the same erased term with the same retained assertions and packet operations. Conjunction is associative, commutative, and idempotent; result splitting is Proposition 2.4. The common source origin identifies the implementation, and Proposition 3.4 identifies the observation questions. Proof availability determines which work is enabled while preserving the successful-execution predicate. $\square$

Corollary 5.11 (Monotone certified refinement). *Normalization by a well-evidenced evaluator followed by source conjunction can only restrict completed observations on the retained interface. Semantic contradictions persist. A singleton observation can remain unchanged or become inconsistent. Checked rejections persist at their retained scopes; pending obligations may acquire evidence.*

Proof. State preservation gives equality for normalization and inclusion for source conjunction. Theorem 2.7 transports inclusion through the fixed projections and quantifiers. Subsets of the empty set remain empty, and subsets of a singleton contain no different atom. Rejection premises persist by Lemma 3.1. $\square$

6 Surface language and calling protocols

The surface constructs elaborate into the predicates, proof obligations, and packet operations already defined. Function syntax writes `func(parameters) -> T: e` for an implementation and `func(parameters) -> T` for an interface. The result annotation creates a body obligation. The implementation's protocol identifies its slots and their permitted bindings.

6.1 Universal declarations and instances

The form `forall (A, B) T` binds semantic type variables in T ; $A: U$ specifies an upper bound. A single unbounded variable may omit parentheses. Quantifiers have low precedence, with parentheses delimiting quantified arguments and results. An outer universal has the equivalent forms

```
map(A, B): func(func(A) -> B, [...A]) -> [...B]
map: forall (A, B) func(func(A) -> B, [...A]) -> [...B]
map: func<A, B>(func(A) -> B, [...A]) -> [...B]
```

Each describes one subject at every instance. A lexical prefix inside a record abbreviates a quantifier around that record:

```
r: {
  forall A
    id: func(A) -> A
}
r: forall A {id: func(A) -> A}
```

The binder's private scope is retained when another file contributes constraints to the record.

A parametric alias `Box(A) = {value: A}` constructs a type expression by substitution. A quantified field `box(A): {value: A}` constrains one subject by every instance. Ordinary `let` names an expression. Instantiation `f[T]` records ALL-E on the same subject, retaining its other constraints and its instance expression. It erases to `f`. Contextual binder forms are selected by the language version, and elaboration preserves scope, source roots, and closure origins through copying and export.

6.2 Arguments, omission, and partial application

The parameter forms use the packet syntax of proposal 4484 [5]:

Form	Binding
$a: \quad T$	Position or label a
$a!: \quad T$	Required label a
$a?: \quad T$	Optional label a ; omission is absence
$_ \sim x: \quad T$	Position only; x is a body alias
T or $_:$ T	Anonymous positional slot

Positional-only slots precede named slots, which precede name-only slots. Labels identify original slots. Duplicate bindings are malformed even when the supplied values agree. Hidden labels include package identity, and attributes remain attached metadata.

An omission default $= d$ is an executable part of the protocol, with a checking obligation in its declaration environment. It supplies an argument when the slot is omitted. An explicitly supplied description, including $_$, suppresses the omission default and retains its own CUE preferences. A required-label slot with a default is omissible because the protocol supplies its value. An optional slot represents absence when omitted.

A packet interface includes all accepted bindings and omissions. Attachment checks inclusion of these packet sets contravariantly. An adapter that changes a label or adds a default is an explicit new closure. An open interface such as `func(a: int, ...) -> number` contains an ordinary completion variable for the remaining slots. Linking an implementation supplies its row, and invocation uses the resulting protocol evidence.

A call ending in $\dots$ supplies some slots and returns a closure over the remainder. Supplied arguments are checked immediately; the body runs when the packet is complete. Normalize partial bindings to original slot identities, retaining unbound defaults. If b is a fixed binding and q a residual packet, let $b \oplus q$ be their valid combined packet and put $A_b = \{q \mid b \oplus q \in A\}$.

Proposition 6.1 (Partial application). *If $f : A \rightarrow B$ and $f_b(q)$ executes $f(b \oplus q)$, then $f_b : A_b \rightarrow B$. Slot substitution transports the protocol certificate. Successive binding of disjoint slots and simultaneous binding give the same normalized closure.*

Proof. A successful residual call is an original call with packet in A , so its result belongs to B . Slot normalization records the same map from original slots to supplied values in either grouping. $\square$

6.3 Foreign calls and finite traversal

A foreign bridge supplies a certified closure whose public interface describes the checked adapter. The adapter retains representation checks, conversions, and output validation. An arbitrary-precision integer interface may consequently have native range failures while preserving its successful-result contract. Logical kinds are checked at attachment independently of native widths. Erased type evidence leaves the adapter's conversion path intact [6].

A finite-list fold is specified by

$$\text{fold}(s, z, []) = z, \quad \text{fold}(s, z, x :: xs) = \text{fold}(s, s(z, x), xs).$$

Its certificate is $\forall A, B. [[B, A] \rightarrow B, B, \text{List}(A)] \rightarrow B$. The equations define traversal by finite list length, with callback evidence supplied at each step. User-defined calls retain the base cycle discipline, and all higher-order evaluation is subject to accounted work [4].

7 Examples

Each annotated declaration or call is a conformance check in its indicated scope. Negative cases are checked separately from successful ones. Utility definitions are reused only after they are displayed. Repeated declarations shown as constraints on one subject are unified. Commented future refinements are separate extensions of the initial configuration.

Evaluation results denote summaries with their source dependencies retained. The comments distinguish constraint failure, rejected certification, and pending work. Bodyless interfaces are hypotheses until their implementations are linked.

7.1 Quantified subjects

We first distinguish completing a description from imposing every type instance on one subject.

Example 7.1 (Completion witnesses and universal subjects).

```
#Cell: {value: int}
left: #Cell & {value: 1}
right: #Cell & {value: 2}

empty(A): [...A]           // the empty list
impossible(A): A           // _|_, using A = _|_
Box(A) = {value: A}
one: Box(int) & {value: 1}
box(A): {value: A}         // _|_: one required field at every A
```

The copies of `#Cell` choose separate witnesses. Each quantified field instead retains one subject across every instance. Instantiation at bottom forces `empty` to be the empty list and refutes `impossible` and `box`. The parametric alias constructs the particular record predicate used by `one`.

Example 7.2 (One identity at many instances).

```
id(A): func(x: A) -> A: x
integer: id(7)           // 7
text: id("seven")        // "seven"
record: id({port: 443})  // {port: 443}
chosen: id[int & >=0](7)  // 7
wrong(A): func(x: A) -> A: 0 // rejected: 0 is not arbitrary A
```

The same closure realizes the integer, string, and record instances. Explicit instantiation records a bound proof without changing that closure. The constant body in `wrong` is refuted at a singleton type containing a value other than zero: its successful return violates that instance.

Example 7.3 (Common predicates and preserved refinements).

```
pair(A: number): func(x: A, y: A) -> [A, A]: [x, y]
p: pair[int & >=1](2, 5) // [2, 5]
q: pair[1, 2.5]         // [1, 2.5]
bad: pair[int](1, 2.5)   // rejected: second argument is not int
swap(A, B): func(p: [A, B]) -> [B, A]: [p[1], p[0]]
s: swap([7, "seven"])   // ["seven", 7]
```

The shared variable denotes a predicate containing both arguments. For the mixed numeric pair, $A = \{1, 2.5\}$ is admissible. The explicit integer instance excludes the fractional argument. Two variables in `swap` retain the independent component predicates; the result exchanges their positions.

Example 7.4 (Bounded selection).

```
min(A: number): func(x: A, y: A) -> A: {
  if x <= y {out: x}
  if x > y {out: y}
}.out
m: min[int & >=10](12, 20) // 12
same(A: number): func(x: A) -> A: x + 1 // rejected
```

The bound permits comparison, and each branch returns a value already known to inhabit A . This preserves every numeric subtype, including the stated interval. Addition has a different result obligation: $A = \{0\}$ refutes `same`, since its successful result is 1.

Example 7.5 (Distinct universal ranges).

```

choose(A: int):    func(A, A) -> A
choose(A: string): func(A, A) -> A
stronger(A: int | string): func(A, A) -> A

r(A): {id: func(A) -> A}
r: {label: "utilities"}
// Equivalent accumulated predicate:
r(A): {id: func(A) -> A, label: "utilities"}

```

The first two clauses require behavior on subtypes of integers and on subtypes of strings. The third also includes mixed subtypes of their union. The label field is independent of the binder, so equation (9) moves it under the universal while preserving its dependency annotation.

7.2 Higher-order and impredicative use

The same quantifier rules apply inside callback parameters, returned interfaces, and type arguments.

Example 7.6 (A polymorphic map).

```

map(A, B): func(g: func(A) -> B, xs: [...A]) -> [...B]: [
  for x in xs {g(x)}
]
inc: func(x: int) -> int: x + 1
text: func(x: int) -> string: "\(x)"
numbers: map(inc, [1, 2, 3]) // [2, 3, 4]
strings: map(text, [1, 2, 3]) // ["1", "2", "3"]
invalid: map[int, string](inc, [1]) // rejected: callback result

```

With A, B rigid, each list element has type A and each successful callback return has type B . The result is therefore a list of B . The final call requires an integer-to-string callback; `inc` cannot establish that result contract.

Example 7.7 (Filtering and nested traversal).

```

filter(A): func(p: func(A) -> bool, xs: [...A]) -> [...A]: [
  for x in xs if p(x) {x}
]
partition(A): func(p: func(A) -> bool, xs: [...A]) -> {
  yes: [...A], no: [...A]
}: {
  yes: [for x in xs if p(x) {x}]
  no: [for x in xs if !p(x) {x}]
}
positive: func(x: int) -> bool: x > 0
kept: filter(positive, [-2, 0, 4]) // [4]
parts: partition(positive, [-2, 0, 4]) // yes: [4], no: [-2, 0]
flatMap(A, B): func(g: func(A) -> [...B], xs: [...A]) -> [...B]: [
  for x in xs for y in g(x) {y}
]
duplicate(A): func(x: A) -> [A, A]: [x, x]
twice: flatMap[int, int](duplicate[int], [1, 2]) // [1, 1, 2, 2]

```

Filtering and partitioning select existing elements and preserve their arbitrary type. The two partition comprehensions use a pure predicate. In `flatMap`, each outer call produces a list of B , whose elements populate the final list. The explicit instance of `duplicate` supplies the required callback.

Example 7.8 (Mapping a record).

```

mapPoint(A, B): func(g: func(A) -> B, p: {x: A, y: A}) -> {
  x: B, y: B
}: {x: g(p.x), y: g(p.y)}
p: mapPoint(text, {x: 2, y: 3}) // {x: "2", y: "3"}

```

The parameter guarantees both selected fields. Width subtyping permits further argument fields, and the two callback applications establish the two stated result fields. This is componentwise certification over the shared argument record.

Example 7.9 (Composition and captured data).

```

compose(A, B, C): func(g: func(B) -> C, f: func(A) -> B) ->
    func(A) -> C: func(x: A) -> C: g(f(x))
pipeline: compose[int, int, string](text, inc)
answer: pipeline(4) // "5"
withPrefix: func(prefix: string) -> func(string) -> string:
    func(s: string) -> string: prefix + s
warn: withPrefix("warning: ")
message: warn("retry") // "warning: retry"

```

The body of `compose` is the derivation in Theorem 5.9: an A -value becomes a B -value and then a C -value on success. The second example captures an ordinary string. Its interface is formed independently of the particular captured prefix.

Example 7.10 (A universally quantified parameter).

```

useBoth: func(p: forall A func(A) -> A) -> [int, string]: [
    p[int](7), p[string]("seven"),
]
ok: useBoth(id) // [7, "seven"]
onlyInt: func(x: int) -> int: x
bad: useBoth(onlyInt) // rejected: universal callback required

```

The parameter itself must satisfy a universal interface. Its two instances are available inside one body. Linking `id` discharges that hypothesis; an integer-only implementation cannot supply the string-instance capability.

Example 7.11 (Quantified results and nested rank).

```

constant(A): func(x: A) -> (forall B func(B) -> A):
    func(_) -> A: x
seven: constant[int](7)
a: seven[string]("ignored") // 7
b: seven[bool](true) // 7

consume: func(k: func(forall A func(A) -> A) -> int) -> int: k(id)
use: consume(func(p: forall A func(A) -> A) -> int: p[int](8)) // 8

```

The returned closure captures the fixed A -value and is uniform in the later argument type B . The result quantifier commutes with the outer arrow by Proposition 2.4. The final declaration places a universal inside a callback's parameter, and checks that callback before invocation.

Example 7.12 (An impredicative instance).

```

let Identity = forall A func(A) -> A
again: id[Identity](id)
a: again[int](3) // 3
b: again[string]("three") // "three"
ids: [...Identity]
ids: [id, again]
c: ids[0][bool](true) // true
bad: id[Identity](onlyInt) // rejected: onlyInt lacks Identity

```

The instance `Identity` is a finite type expression containing a universal. It belongs to the same semantic sort as other instances. Applying `identity` to `id` consequently returns the same polymorphic closure, which can also be stored in a list. The monomorphic `onlyInt` lacks the required evidence.

7.3 Live descriptions and accumulating evidence

Open interfaces use proved bounds while retaining the source references that future unification may constrain.

Example 7.13 (A live input and a fixed alias).

```
let Fixed = int
fixed: func(x: Fixed) -> int: 2 * x

Live: int
live: func(x: Live) -> int: 2 * x
a: fixed(2)           // 4
b: live(2)            // 4

// A later file may add Live: >=0.
```

Both body proofs use the upper bound `int`. The alias has a fixed denotation; the field retains a live description coordinate. Its call uses a fresh packet and keeps the reference to `Live`, so further constraints on that field continue to validate the invocation.

Example 7.14 (Input and result constraints on one call).

```
_input: int
_output: int
_f: func(x: _input) -> int: 2 * x
a: _f(2)
a: _output           // 4, with both dependencies retained

// Separate possible refinements:
// _input: >=0        // preserves a: 4
// _input: >2         // _|_ at the invocation
// _output: >=4       // preserves a: 4
// _output: <4        // _|_ at the result
```

Each commented addition is a separate refinement. The input restriction remains attached to the activation, and the output restriction concerns that call's result cell. A compatible addition preserves 4; an incompatible one refutes the corresponding demand. The reusable implementation retains its integer result contract.

Example 7.15 (A stronger bound completes a proof).

```
A: number
f: func(x: A) -> int: x      // body goal pending

// A later file adds:
A: int                     // body goal can now be proved
answer: f(2)               // 2
```

The initial domain has only a numeric upper bound. A fractional point in that bound is not a witness in every live domain. The body obligation remains pending until the added integer bound proves $A \preceq \text{int}$. A fixed `number` domain would instead admit the persistent counterexample 1.5.

Example 7.16 (Proving a live result).

```
R: int
same: func(x: R) -> R: x      // certified
constant: func(_: int) -> R: 2 // result goal pending
checked: func(x: int) -> R: x & R // certified partial body

// Refining R: 2 proves the constant's result goal.
// Refining R: >2 refutes that constant result.
```

The variable return supplies R directly. Proving the constant result requires $\{2\} \preceq R$, which its upper bound alone does not establish. The assertion in `checked` establishes membership on successful execution. Refinement to the singleton supplies the constant's proof; a predicate excluding 2 supplies its countercertificate.

Example 7.17 (Evaluation produces certification premises).

```
seed: func(x: int) -> int: x + 1
k: seed(2)
shift: func(x: int) -> int: x + k
limit: shift(1)
bounded: func(x: int) -> int: x & <=limit
a: bounded(3) // 3; limit is 4
```

The certificate for `seed` supplies a result bound for `k`. That bound supports the body of `shift`, whose call computes `limit`. The last assertion then finishes its data check. Bounds can precede concrete values, but each retains the invocation evidence and validation demands supporting it.

Example 7.18 (A partially known capture).

```
factor: int
scale: func(x: int) -> int: factor * x // certified with a live capture
factor: 3
answer: scale(4) // 12

_f: {
  z!: int
  f: func(x: int, y: int) -> int: x + y + z
}
bar: (_f & {z: 10}).f(100, 1000) // 1110
```

The multiplication proof uses the integer bound on `factor`. Completing the multiplication also needs its value. Once that value is supplied the result is 12, with the capture dependency retained. A later contradiction in the capture therefore remains a contradiction of the demanded computation. The required capture in `_f` likewise supplies a body-proof premise before it supplies a runtime value. The meet completes a fresh instance whose method sees `z: 10`; it does not fill the prototype. A call through the unrefined prototype remains incomplete. Separate instances retain independent captures. Source export may share their code origin, but must instantiate each live capture environment in its lexical record so later copying and refinement continue to rebind it.

Example 7.19 (An unfounded proof dependency).

```
k: f(0)
f: func(x: int) -> int: k
a: f(1)
```

The result annotation of `f` is a goal. Using that goal to type its own pending call would merely repeat the original obligation. The proof dependency remains residual. An independent `k: int` can establish the body theorem; obtaining a value for the cyclic computation still requires its own validation.

Example 7.20 (A live observation guard).

```
limit: int
f: func(x: int & >limit & <10) -> _|_: _|_
// Pending: the result is empty; the domain depends on limit.

// Separate possible refinements:
// limit: 10 // empty domain: relevance demand discharged
// limit: 0 // witness x = 1: relevant interface rejected
```

The empty result activates an input-emptiness demand. With `limit` unknown, the store admits both empty and inhabited regions. The first refinement proves emptiness; the second gives the robust witness 1. The latter is a relevant admission rejection, while the failing closure remains a semantic inhabitant of its partial arrow.

Example 7.21 (Coverage requires the opposite inclusion).

```
Small: int
local: func(x: Small) -> int: x + 1
consumer: func(g: func(int) -> int) -> int: g(2)
a: consumer(local) // coverage goal pending
```

The callback requirement is $\text{int} \preceq \text{Small}$. The field supplies $\text{Small} \preceq \text{int}$, so coverage remains pending even though a particular packet can be constrained successfully. The fixed alias `let Small = int` would supply both directions.

Example 7.22 (A live universal bound).

```

Upper: number
choose(A: Upper): func(x: A, y: A) -> A: {
  if x <= y {out: x}
  if x > y {out: y}
}.out
a: choose[Upper](2, 5)           // 2, with packet demands
needs: choose[int & >=0](2, 5)   // instance-bound goal pending
// A later file may add Upper: int; a still has result 2.

```

The body uses $A \preceq \text{Upper} \preceq \text{number}$ for comparison and returns an input. Instantiation at `Upper` has a reflexive bound proof, while the packet retains its memberships. The explicit nonnegative-integer instance needs its own inclusion into the live bound. Refining an upper approximation does not establish that reverse inclusion.

Example 7.23 (Residual validation and strict arguments).

```

_x: int
_f: func(x: _x) -> int: 2 * x
a: 4 & _f(2)           // a valid residual

ignore: func(_: int) -> int: 7
pending: ignore(_)     // demanded argument is incomplete
bad: ignore(_|_)       // _|_, never 7

```

The first residual retains the equation that validates the original call. In the other calls, strictness preserves the argument demand even though the body returns a constant. An incomplete demanded operand remains incomplete; a failed one propagates failure.

Example 7.24 (Scoped instances and linking hypotheses).

```

id(A): func(x: A) -> A: x
first: id[int](3)
second: id[string]("three")
use: func(p: forall A func(A) -> A) -> int: p[int](4)
result: use(id)           // 4

```

Each instance has its own proof record while denoting the same implementation. The callback hypothesis used to certify `use` is discharged by the supplied universal certificate. Neither instantiation changes a shared type cell to satisfy an inclusion nor a local hypothesis proves its own eventual linking.

7.4 Conjunctive and relevant interfaces

An intersection constrains one implementation. Its explicit observations determine the relevant admission questions.

Example 7.25 (A shared quantified record).

```

utilities(A): {
  id: func(x: A) -> A: x
  wrap: func(x: A) -> {value: A}: {value: x}
}
utilities: {name: "core"}
wrapped: utilities.wrap[string]("payload") // {value: "payload"}

// interface.cue
wrap(A): func(A) -> {value: A}
// metadata.cue
wrap(B): func(B) -> {tag: "wrapped"}
// implementation.cue
wrap(T): func(x: T) -> {value: T, tag: "wrapped": {
  value: x, tag: "wrapped"
}}

```

The fields of `utilities` retain the record's common witness when selected. The separate `wrap` declarations align by alpha-renaming. One implementation then supplies both the value field and the metadata field at each instance, using the conjunction law for common-domain results.

Example 7.26 (Contravariant attachment).

```

wide: func(number) -> string
wide: func(x: int) -> string: "ok"           // rejected: domain coverage
narrow: func(int) -> string
narrow: func(x: number) -> string: "ok"      // accepted
apply: func(g: func(number) -> string) -> string: g(1.5)
ints: func(x: int) -> string: "ok"
bad: apply(ints)                             // rejected at linking

```

The first attachment requires $\text{number} \preceq \text{int}$, which the fractional packet 1.5 refutes. The second requires the valid inclusion $\text{int} \preceq \text{number}$. The higher-order call makes the same coverage check at linking, before its client uses the promised numeric capability.

Example 7.27 (Regions of a conjunctive interface).

```

convert: (func(int) -> string) & (func(string) -> int)
refine: (func(number) -> number) & (func(int) -> int)
choice: (func(int) -> int) | (func(int) -> bool)
classify: func(x: int) -> ("negative" | "nonnegative"): {
  if x < 0 {out: "negative"}
  if x >= 0 {out: "nonnegative"}
}.out
classify: (func(int & <0) -> "negative") &
          (func(int & >=0) -> "nonnegative")
c: classify(-3)                             // "negative"

```

The two `convert` clauses constrain disjoint input regions of one closure. The clauses of `refine` both apply on integers. The body tests of `classify` establish the finer regional results; those tests are executable code. The interface conjunction preserves both obligations without choosing an implementation.

Example 7.28 (Explicitly contradictory observations).

```

f: (func(int) -> int) & (func(int) -> bool) // rejected by relevance
g: func(int) -> _|_                        // rejected by relevance
h: (func(number) -> int) & (func(int) -> bool) // rejected on int
three: (func(int) -> (0 | 1)) &
       (func(int) -> (1 | 2)) &
       (func(int) -> (0 | 2))              // rejected jointly
callback: func((func(int) -> int) &
              (func(int) -> bool)) -> string // rejected in parameter
alternative: (func(int) -> _|_) |
             (func(int) -> int)             // first arm rejected under its guard

```

The first two interfaces have a fixed inhabited integer region with empty result. The three-clause example needs the joint result intersection; pairwise checks would miss it. Traversal also reaches callback parameters. The first union arm is rejected under its own guard: that diagnostic neither proves the arm's semantic emptiness nor unconditionally rejects the alternative.

Example 7.29 (An independent anchor for a universal).

```

id(A): func(A) -> A
id: func(int) -> int           // redundant valid instance
quiet(A): func(A) -> A
quiet: func(int) -> bool       // rejected by the integer anchor

```

The integer clause is an independent source observation. Its domain anchors a checked integer instance of the universal clause. This is consistent for `id`; for `quiet`, the two results become $\text{int} \wedge \text{bool}$. The same anchor survives floating the independent clause beneath the binder.

Example 7.30 (Generic meet and explicit specialization).

```

meet(A, B): func(x: A, y: B) -> (A & B): x & y
good: meet({a: 1}, {b: true})      // {a: 1, b: true}
bad: meet[int, bool](1, true)      // _|_
specialized: func(int, bool) -> (int & bool) // rejected interface
f(A): func(xs: [A, A]) -> A: xs[0] & xs[1]
x: f([a: 1], [b: 1])              // {a: 1, b: 1}
y: f([1, 2])                      // _|_

```

The generic result $A \wedge B$ has no uniform rigid refutation, and `meet` certifies its successful common completion. The integer–Boolean call fails. Writing that instance as `specialized` creates a separately rejected observation. Open-record operands share their completion before projection; the scalar operands in the last call have incompatible completions.

Example 7.31 (Annotations and executable assertions).

```

asText: func(x: int) -> string: x      // rejected
asTwo:  func(x: int) -> 2: 1           // rejected
positive: func(x: int) -> (int & >0): x // rejected
positiveChecked: func(x: int) -> (int & >0): x & >0
ok: positiveChecked(3)                 // 3
bad: positiveChecked(0)                 // _|_
pick: func(x: int | string) -> int: x  // rejected
pickChecked: func(x: int | string) -> int: x & int
stop(A): func(x: A) -> A: _|_          // certified partial function

```

Each annotation asks the body to establish its result. A retained assertion can instead enforce that result conditionally on success. The positive and integer-selecting checks therefore have partial certificates and may fail on particular inputs. The generic `stop` is a closure whose invocation fails; its uniform result obligation is satisfied by partial correctness.

Example 7.32 (Equivalent arrangements of constraints).

```

one: {f: func(x: int) -> (int & >0): x & >0}
two: {
  f: func(int) -> (int & >0)
  f: func(x: int) -> _: x & >0
}
three: {
  f: func(x: int) -> int: x & >0
  f: func(int) -> >0
}
four: {
  f: func(int) -> int
  f: func(x: int) -> >0: x & >0
}
five: {
  f: func(int) -> int
  f: func(int) -> >0
  f: func(x: int) -> _: x & >0
}

```

Each record is an independent subject with one implementation origin. Its accumulated interface is $\text{int} \rightarrow (\text{int} \wedge >0)$, and its erased body contains the same positive-result assertion up to origin renaming. Keeping the body derivation supports each arrangement regardless of an intermediate weaker annotation. Each succeeds on 3 and fails on 0.

Example 7.33 (Field evidence and presence).

```

get: func(r: {a: int}) -> int: r.a
ok: get({a: 1, b: true})      // 1
bad: func(r: {a: int}) -> int: r.b // rejected: no field evidence
needsB: func(r: {a: int, b: bool}) -> int: r.a
missing: needsB({a: 1})       // pending: b is unresolved
absent: {a?: int & bool}      // a must be absent
conflict: {a: int & bool}     // _|_
fixed: [int, string] & [int]  // _|_: length conflict

```

A guaranteed field supports selection; width subtyping admits extra fields. The body selecting `b` has no guaranteed field evidence. The call to `needsB` retains a demand for the unresolved `b` field. Empty optional contents force absence, empty required contents refute the record, and fixed tuples of unequal length conflict.

Example 7.34 (Failure inherited by composition).

```
ignore: func(_: _|_) -> int: 7
x: ignore(_|_)           // _|_, never 7
bounded: func(x: int) -> int: x & >0
pipe: compose[int, int, string](text, bounded)
good: pipe(3)           // "3"
bad: pipe(0)            // _|_, inherited from bounded
```

Strict application cannot obtain the constant result from a failed argument. The pipeline has the stated partial type, but its explicit positivity check fails at zero. On inputs satisfying the success premises of Theorem 5.9, the same derivation guarantees successful composition.

7.5 Calling protocols and implementation identity

Packet binding describes actual calling behavior, and closure equality preserves the identity of the implementation being constrained.

Example 7.35 (Slot binding and omission).

```
sum: func(a: int, b: int) -> int: a + b
p: sum(2, 3)           // 5
q: sum(a: 2, b: 3)     // 5
r: sum(2, b: 3)        // 5
key: func(a!: int, note?: string) -> int: a
k: key(a: 4)           // 4; note absent
n: key(a: 4, note: "audit") // 4
badKey: key(4)         // rejected: a is name-only
scale: func(_~value: int, _: int) -> int: value * 2
s: scale(5, 99)        // 10
badLabel: scale(value: 5, 99) // rejected: label and order
bad1: sum(1, a: 1)      // duplicate binding
bad2: sum(1, 2, 3)      // surplus argument
bad3: sum(a: 1, b: 2, c: 3) // unknown label
bad4: sum(1)           // missing argument
pending: sum(_, 2)      // supplied but incomplete
```

Position and label resolve to original slots before value constraints are checked. Duplicate bindings therefore remain invalid even when equal. A body-only alias introduces no callable label. Optional fields require presence evidence before selection; a supplied unknown operand is a present slot whose value is still demanded.

Example 7.36 (Package-qualified labels).

```
package internal
measure: func(_value: int @go(Int), unit!: string) -> int: _value
inside: measure(_value: 7, unit: "ms")
// Other packages can supply the positional slot, not this hidden label.
```

The hidden label carries its declaring package identity. Inside that package it can be supplied by name; the positional slot remains available externally. The attribute belongs to the descriptor as metadata and leaves ordinary binding unchanged.

Example 7.37 (Defaults belong to the protocol).

```
add: func(a: int, b: int = 10) -> int: a + b
x: int | *5
omitted: add(1)           // 11
supplied: add(1, x)        // 6
unknown: add(1, _)        // incomplete
render: func(text: string, suffix!: string = "!") -> string:
  text + suffix
hello: render("hello")    // "hello!"
question: render("hello", suffix: "?") // "hello?"
badDefault: func(x: int = "x") -> int: x // rejected
```

Omission supplies the declared default. A supplied argument retains its own preference and suppresses that default, even while incomplete. The required-label slot in `render` is omissible because its protocol supplies a value. The final default conflicts with its parameter constraint at declaration.

Example 7.38 (Adapters are implementations).

```

raw: func(_~n: int) -> int: n + 1
named: func(x: int) -> int: raw(x)
configured: func(x: int = 2) -> int: raw(x)
a: named(x: 2) // 3
b: configured() // 3

```

The wrappers introduce the stated label and omission behavior as executable protocols. Each has its own origin and a body calling `raw`. Their certificates describe those actual protocols; attaching a type to `raw` retains its original positional slot.

Example 7.39 (Open rows and normalized partial bindings).

```

T: func(a: int, ...) -> number
f: T & (func(a: int, b: int) -> int: a + b)
x: f(1, 2) // 3
missing: f(1) // rejected: b is required
add3: func(a: int, b: int, c: int) -> int: a + b + c
part: add3(1, ...)
part: func(int, int) -> int
last: part(2, ...)
answer: last(3) // 6
same: add3(a: 1, ...) & part
again: same(2, 3) // 6

```

Linking supplies the open interface's remaining slot, including its requiredness. Each partial application retains the unbound requirements and any defaults. Binding by the original label or by its position yields the same normalized slot map, so the two occurrences constrained by `same` identify the same partial closure.

Example 7.40 (Closure identity through copying).

```

x: {f: func(y: int) -> int: y}
a: x.f & x.f
b: x.f & {x}.f
c: x.f & (x & {g: 1}).f
r: [a(2), b(2), c(2)] // [2, 2, 2]
d: [for v in [1, 2] {func(y: int) -> int: y + v}]
e: d[0] & d[1] // _|_: unequal captures

```

References reached through copying, embedding, and unrelated record refinement preserve the origin and free captures. All three self-unifications therefore remain callable. The comprehension closures share an origin but capture different values, so their singleton predicates have empty intersection.

Example 7.41 (Shared results and implementation alternatives).

```

left: func(x: int) -> {a: int}: {a: x}
right: func(x: int) -> {b: int}: {b: x}
both: func(x: int) -> {a: int, b: int}: left(x) & right(x)
r: both(3) // {a: 3, b: 3}
first: func(x: int) -> int: 1
second: func(x: int) -> int: 2
conflict: first & second // _|_: distinct origins
choice: first | second // two possible implementations
preferred: *first | second
p: preferred(3) // 1 after default selection

```

The body of `both` explicitly combines its two result descriptions and has a new origin. Conjoining the distinct scalar-returning closures instead intersects disjoint singleton predicates. Their disjunction retains both implementations; preference selection chooses an alternative according to the base default rules, with its source dependencies preserved.

7.6 Validation, boundaries, and finite computation

Validation demands and primitive certificates connect the type discipline to completed observations.

Example 7.42 (Validation across field and call boundaries).

```
positive: func(x: int) -> bool: x > 0
#Positive: {value: int, _valid: true & positive(value)}
n: (#Positive & {value: 7}).value // 7
bad: #Positive & {value: -1} // _|_
raw: func(x: number) -> string: "\{x}"
call: {arg: number, result: raw(arg)}
call: {arg: int, result: string} // another file's restriction
ok: (call & {arg: 7}).result // "7"
wrong: (call & {arg: 1.5}).result // _|_
```

The validator’s hidden dependency survives selection of `value`. The two declarations of `call` constrain one existential packet and result while the reusable numeric implementation keeps its capability. The integer restriction excludes the fractional invocation without changing that implementation.

Example 7.43 (Conditional certificates and native checks).

[illegible]

Bodyless function fields supply named hypotheses for client checking; actual implementations discharge them at linking. The explicit assertion retains the positive-result check. A native adapter may reject the large integer at its representation boundary while preserving its portable partial contract. The fractional argument conflicts with the logical integer input.

Example 7.44 (Finite iteration and repeated invocation).

```
fold(A, B): func(step: func(B, A) -> B, seed: B, xs: [...A]) -> B
plus: func(x: int, y: int) -> int: x + y
sum: fold[int, int](plus, 0, [1, 2, 3]) // 6
empty: fold[int, int](plus, 0, []) // 0
twice: func(n: int) -> int: n + n
nested: twice(twice(twice(2))) // 16
```

The fold hypothesis is linked to the finite-list primitive. Its empty case returns the seed, and each nonempty case consumes one list cell after a certified step. Length induction supplies the universal certificate. Repeated nested calls create ordinary finite activations.

Example 7.45 (Direct contradictions and residual equations).

```
emptyInt: int >0 & <1      // _/_: integer interval is empty
emptyData: {a: int} & {a: bool} // _/_
cycle: {a: int & (5 * b), b: int & (a - 1)}
grounded: cycle & {a: 0}    // _/_ by ground propagation
```

The integer interval and the incompatible required field are mandatory refutations. A local propagator can retain the ungrounded arithmetic cycle; a stronger checked arithmetic proof can refute it directly. The ground constraint exposes the inconsistency to elementary propagation. An unresolved residual continues to carry those equations.

Example 7.46 (Native folds and operand evidence).

```
merge(A): func(xs: [A, ...A]) -> A: and(xs)
x: merge([a: 1], {b: 1}) // {a: 1, b: 1}
choose(A): func(xs: [...A]) -> A: or(xs)
y: choose(["ok"])
first(A): func(xs: [...A], i: int) -> A: xs[i]
z: first([1, 2], 1) // 2
```

The guaranteed head of `merge`'s input proves its result type. Replacing `[A, ...A]` by `[...A]` loses that proof because `and([])` returns `_`. Neither the result annotation nor a successful nonempty call repairs the universal claim. The empty cases of `choose` and `first` have no successful result. Their input domains remain admitted by their partial contracts; evaluation still retains any unresolved demand.

8 Related designs

8.1 CUE function signatures

Proposal 4484 and its theory addendum are compared at repository revision a949162aedb8, dated 19 September 2026 [5, 6]. The addendum interprets signatures as call filters: input constraints meet covariantly and calls outside them fail. It identifies modular checkability as the resulting loss. Its proposed coverage component joins required domains while call restrictions meet, and checks an inclusion between the two. It also identifies result-side attachment checking as a separate obligation.

These observations isolate the distinction between a constraint on a call and a capability required from an implementation. A client expecting $\text{number} \rightarrow \text{string}$ can use the packet 1.5. An integer-only implementation fails the coverage goal $\text{number} \preceq \text{int}$, even if every integer call succeeds. Covariant tightening can retain a coherent call filter while accepting that attachment and postponing its failure to invocation. The present attachment judgment makes the capability obligation explicit. Call-local restrictions remain ordinary predicates on existential packet and result cells.

The addendum’s pointwise body conjunction constructs behavior by combining results. The main proposal describes opaque function identity; the two accounts are distinct [5, 6]. Here conjunction intersects predicates on one closure, and application satisfies the inclusion in Proposition 2.5. Explicit adapters provide new protocols or combine result descriptions. A module replacement judgment obtains its orientation from the clients to be preserved, whereas conjunction retains its symmetric logical meaning.

8.2 Set-theoretic types and partial behavior

Semantic subtyping interprets Boolean type structure and arrow intersections extensionally, retaining correlations between inputs and results [11]. Its polymorphic developments combine instantiation, subtyping, and expansion under stated semantic assumptions [12, 13, 14]. CDuce’s polymorphism tutorial illustrates intersections obtained from several instances of a higher-order map [15]. Exact observation regions have the same correlation-preserving structure. Polymorphic inference for dynamic languages provides another development of precise Boolean interfaces [16].

The semantic arrow and the operational evidence interface determine which of those checking rules can be transported here. In particular, a partial arrow with an empty result may have inhabitants. Relevant admission concerns its explicit observations, and invocation evidence concerns its usable protocol. Each imported inference rule must preserve those interpretations and transport the evidence required by eliminations.

Elixir’s strong-arrow account additionally reasons about failure outside a declared input domain, using existing guards and operations at dynamic boundaries [17]. A successful-return contract and an out-of-domain failure property express different facts. This proposal’s arrow states the former; its explicit assertions, source packet constraints, and native adapters retain the checks that determine actual failure.

8.3 Impredicativity and proof-directed checking

Intersection interpretations of erased universals have precedents in realizability semantics of System F [10]. Their inhabitants are fixed operational realizers, and arrows are predicates on those realizers. Full set-function spaces B^A give a different interpretation, to which Reynolds’s set-theoretic obstruction applies [9]. In the present model the powerset carrier is fixed before type interpretation. Unary predicates already force partial identity at singleton instances; relational parametricity concerns relations between interpretations at different instances.

Bidirectional typing makes synthesis and checking obligations explicit [18]. Predicative higher-rank checking has a complete bidirectional account [20], and higher-rank systems with intersection and union types supply further algorithmic precedents [21]. Value-restricted union elimination preserves a shared evaluated subject [19]. Here an impredicative instance is a finite proof input, and open checking additionally records the store facts supporting its inclusions. Search produces checked evidence within an accounted run.

8.4 Predicate logic and information propagation

The adjunctions in Proposition 2.2 are the logical structure identified by Lawvere; Melliès and Zeilberger develop a bifibrational account of the corresponding presheaf construction [7, 8]. They also explain composition of open descriptions. For $R \subseteq X \times Y$ and $S \subseteq Y \times Z$,

$$(S \circ R)(x, z) \iff \exists y. R(x, y) \wedge S(y, z).$$

Equality is the identity relation, and associativity follows by exchanging existential witnesses. In the category of sets and relations, Cartesian juxtaposition is a tensor and equality supplies self-duality: its snake equation is $\exists y, z. (x = y) \wedge (y = z) \wedge (z = w) \iff x = w$. These equations also hold in a definable subcategory closed under the stated operations. Conjunction is the meet in one predicate fiber; juxtaposition combines interfaces.

Propagators accumulate partial information in cells and notify their dependents [22]. Abstract interpretation organizes sound summaries and their fixed-point computation by an information order [23]. The current bounds are summaries of a retained scoped relation, and proof facts are another form of accumulated information. Finite support justifies each consequence, while exact residualization preserves the underlying problem through incomplete search and subsequent source unification.

9 Conclusion

Universal quantification extends the completion interpretation of CUE by constraining one subject at every semantic type instance. On operational closures it yields partial function contracts with contravariant inputs, covariant results, and correlation-preserving intersections.

Open certification proves those contracts relative to a store. Live references retain their meanings while their established bounds become more precise. Evaluation and certification consequently share a proof-producing propagation process, with exact residuals preserving every required dependency. Relevant observations distinguish interface admission from semantic membership, and positive refutation evidence gives stable diagnostics. Soundness, operational safety, and store-relative composition then describe the guarantees of each certified configuration and of its further refinements.

A CUE integration requirements

The semantic and integration requirements in Section 9 of the theory addendum have the following realizations [6].

Directionality. Contracts on implementations and restrictions on individual calls are both predicates combined by intersection. Attachment explicitly identifies the required interface and the supplied implementation, giving the inclusion $A \preceq C$ its orientation. This judgment applies to provided functions, callbacks, and separately linked code without guessing a module-level role. Live inclusions retain their proof obligations.

Stable observations under added constraints. The exact residual invariant preserves the predicate on the full exposed interface. New source constraints intersect that predicate. Corollary 5.11 therefore preserves contradiction and permits an atomic result only to stay the same or become inconsistent. Positive refutation evidence persists in its scope. Default presentation and comprehension dependencies remain governed by the base observation policy and are retained for source refinement.

Input and output invariants across files. A call has existential packet cells and a shared result. Any file may constrain those cells, independently of the reusable implementation contract. Example 7.14 constrains both sides of one call, and Example 7.42 gives a reusable call record. Additional implementation requirements create attachment goals with the same scope and dependency rules.

Callable self-unification. A complete implementation denotes a singleton, whose intersection with itself is unchanged. Its identity uses origin, free captures, and normalized partial bindings. The same closure reached through copying or embedding consequently retains its invocation evidence without a pointer-identity test. Unrelated record refinements preserve that identity. Pending capture equality remains a scoped constraint; Example 7.40 exhibits both equal and unequal cases.

Portable foreign interfaces. The public contract describes the checked adapter. Representation limits remain executable boundary checks with ordinary failure outcomes, so portable logical signatures can remain independent of native widths. The existing conversion path is preserved by proof erasure. Logical input and result obligations are certified separately from representation checks, as in Example 7.43.

B Auxiliary results

The following lemmas make explicit the remaining local arguments used by the surface elaboration and evaluator interface.

Proposition B.1 (Finite-list traversal). *Suppose s is certified at $[B, A] \rightarrow B$, the seed b at B , and a finite list at $\text{List}(A)$ in one store and scope. The left fold has result certificate B . It is failure-free when every encountered step is failure-free and successful when every encountered step is successful.*

Proof. Induct on the list length. The empty case returns the certified seed. The nonempty case first calls s on the accumulator and head. A successful result has type B , so the induction hypothesis applies to the tail. Failure propagates, divergence prevents later failure, and successful finite steps concatenate. Certificate substitution and store weakening transport the argument to every admissible instance and refinement. $\square$

Proposition B.2 (Attachment preserves an inhabitant). *At a completed world, $\{f\} \cap \llbracket T \rrbracket$ is either $\{f\}$ or $\emptyset$. Grounded certified attachment gives the first case and preserves the closure’s erased execution. Symbolic attachment retains its live dependencies.*

Proof. A singleton has only these two subsets. Certificate soundness establishes membership, and erasure retains the code and captures. The residual invariant preserves the source constraints independently of the executable proof object. $\square$

Proposition B.3 (Contextual residual equivalence). *Equal graph predicates on a common exposed interface give equal semantic observations after any common finite sequence of source conjunctions and semantic projections. The property is preserved by a common fresh context extension.*

Proof. Conjoining the same predicate preserves equality. Existential projection, universal projection, and inverse image are functions on predicates and preserve equality. Induct on the sequence; Proposition 2.2 handles reindexing along the context extension. A coordinate remains exposed until all source constraints addressing it have been applied. $\square$

Proposition B.4 (Grounding a dependency cycle). *A finite set of goals whose proposed derivations each require another goal in the set supplies no grounded certificate by those dependencies alone. Grounding requires independent evidence or a checked inference discharging the relevant hypotheses.*

Proof. Follow a required dependency of a purported grounded root within the set. Without an independent leaf or discharge rule, this path stays in the finite set and repeats a goal. It would therefore require a cyclic proof graph, contrary to well-foundedness. A checked hypothesis-discharge inference supplies a finite derivation rather than such a dependency cycle. $\square$

References

- [1] CUE Authors. *The CUE Language Specification*. <https://cuelang.org/docs/reference/spec/>. Accessed 28 September 2026.
- [2] CUE Authors. *The CUE Language Specification: References and scopes*. <https://cuelang.org/docs/reference/spec/#references>. Accessed 28 September 2026.
- [3] CUE Authors. *Specifying a default value for a field*. <https://cuelang.org/docs/howto/specify-a-default-value-for-a-field/>. Accessed 27 September 2026.
- [4] CUE Authors. *Reference cycles*. <https://cuelang.org/docs/tour/references/cycle/>. Accessed 27 September 2026.
- [5] Marcel van Lohuizen. *Proposal: functions in CUE*. 10 July 2026. Repository revision a949162aedb8. <https://github.com/cue-lang/proposal/blob/a949162aedb86661183c46abef47919d16605014/designs/language/4484-functions.md>.
- [6] Marcel van Lohuizen. *Why CUE Functions Tighten Instead of Widen*. 19 September 2026. Same repository revision. <https://github.com/cue-lang/proposal/blob/a949162aedb86661183c46abef47919d16605014/designs/language/4484-theory-addendum.md>.
- [7] F. William Lawvere. Adjointness in foundations. *Dialectica*, 23(3–4):281–296, 1969. Reprinted in *Reprints in Theory and Applications of Categories*, 16:1–16, 2006. <https://tac.mta.ca/tac/reprints/articles/16/tr16abs.html>.
- [8] Paul-André Melliès and Noam Zeilberger. A bifibrational reconstruction of Lawvere’s presheaf hyperdoctrine. 2016. <https://arxiv.org/abs/1601.06098>.

- [9] John C. Reynolds. Polymorphism is not set-theoretic. In *Semantics of Data Types*, LNCS 173, pp. 145–156, 1984. https://doi.org/10.1007/3-540-13346-1_7.
- [10] Paolo Pistone. On completeness and parametricity in the realizability semantics of System F. *Logical Methods in Computer Science*, 15(4):6, 2019. [https://doi.org/10.23638/LMCS-15\(4:6\)2019](https://doi.org/10.23638/LMCS-15(4:6)2019).
- [11] Alain Frisch, Giuseppe Castagna, and Véronique Benzaken. Semantic subtyping: Dealing set-theoretically with function, union, intersection, and negation types. *Journal of the ACM*, 55(4), article 19, 2008. <https://doi.org/10.1145/1391289.1391293>.
- [12] Giuseppe Castagna and Zhiwu Xu. Set-theoretic foundation of parametric polymorphism and subtyping. In *ICFP*, pp. 94–106, 2011. <https://doi.org/10.1145/2034773.2034788>.
- [13] Giuseppe Castagna, Kim Nguyen, Zhiwu Xu, Hyeonseung Im, Serguei Lenglet, and Luca Padovani. Polymorphic functions with set-theoretic types. Part 1: Syntax, semantics, and evaluation. In *POPL*, pp. 5–17, 2014. <https://doi.org/10.1145/2535838.2535840>.
- [14] Giuseppe Castagna, Kim Nguyen, Zhiwu Xu, and Pietro Abate. Polymorphic functions with set-theoretic types. Part 2: Local type inference and type reconstruction. In *POPL*, pp. 289–302, 2015. <https://doi.org/10.1145/2676726.2676991>.
- [15] CDuce Authors. *Polymorphism in CDuce (development only)*. https://www.cduce.org/tutorial_polymorphism.html. Accessed 28 September 2026.
- [16] Giuseppe Castagna, Mickaël Laurent, and Kim Nguyen. Polymorphic type inference for dynamic languages. *Proceedings of the ACM on Programming Languages*, 8(POPL), 2024. <https://doi.org/10.1145/3632882>.
- [17] José Valim. *Strong arrows: a new approach to gradual typing*. Elixir project, 20 September 2023. <https://elixir-lang.org/blog/2023/09/20/strong-arrows-gradual-typing/>.
- [18] J. Dunfield and N. R. Krishnaswami. Bidirectional typing. *ACM Computing Surveys*, 54(5), 2021. <https://doi.org/10.1145/3450952>.
- [19] J. Dunfield. Untangling typechecking of intersections and unions. *Electronic Proceedings in Theoretical Computer Science*, 45:59–70, 2011. <https://doi.org/10.4204/EPTCS.45.5>.
- [20] J. Dunfield and N. R. Krishnaswami. Complete and easy bidirectional typechecking for higher-rank polymorphism. In *ICFP*, pp. 429–442, 2013. <https://doi.org/10.1145/2500365.2500582>.
- [21] Shengyi Jiang, Chen Cui, and Bruno C. d. S. Oliveira. Bidirectional higher-rank polymorphism with intersection and union types. *Proceedings of the ACM on Programming Languages*, 9(POPL), article 71, pp. 2118–2148, 2025. <https://doi.org/10.1145/3704907>.
- [22] Alexey Radul and Gerald Jay Sussman. *Revised Report on the Propagator Model*. 2011. <https://groups.csail.mit.edu/mac/users/gjs/propagators/>.
- [23] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, pp. 238–252, 1977. <https://www.di.ens.fr/~cousot/COUSOTpapers/POPL77.shtml>.